AMA 564 Deep Learning

2026 Spring

Lecture 2

Feedforward Neural Networks (Multi-Layer Perceptrons)

- The architecture of a MLP is expressed as a composition of a series of functions

$$f_\theta(x) = \mathcal{A}_L \circ \sigma \circ \mathcal{A}_{L-1} \circ \sigma \circ \cdots \circ \sigma \circ \mathcal{A}_1(x), \quad x \in \mathbb{R}^{d_0}$$

where

$$\mathcal{A}_i(x) = W_i x + b_i, \qquad x \in \mathbb{R}^{d_{i-1}}$$

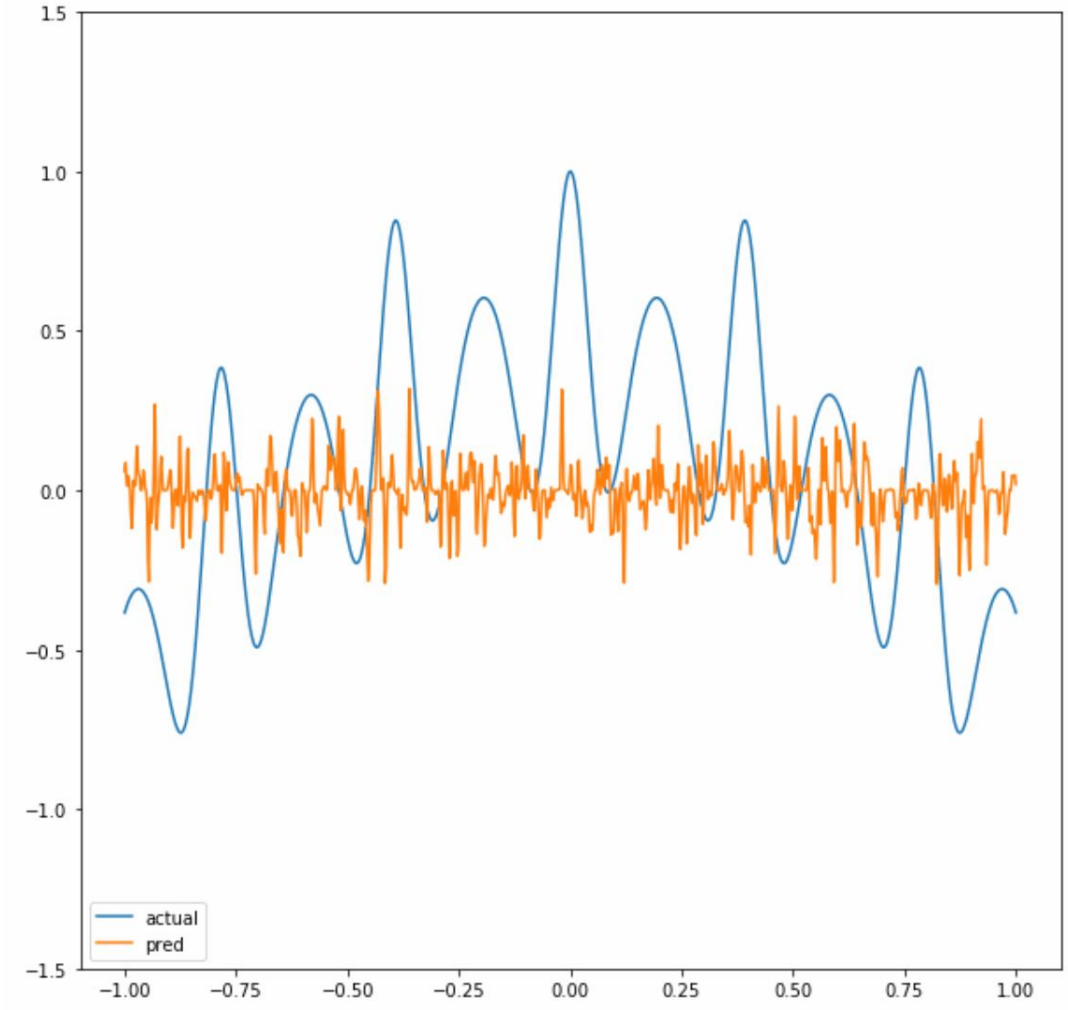
is the i-th linear transformation with

weight matrix $W_i \in \mathbb{R}^{d_i \times d_{i-1}}$ and bias vector $b_i \in \mathbb{R}^{d_i}$,

and σ is the activation function,

e.g., ReLU $\sigma(x) = \max\{x, 0\}$. Sigmoid $\sigma(x) = (1 + e^{-x})^{-1}$.

Universality of Neural Networks



Neural networks can approximate regression functions very well.

Universality of Neural Networks (Arbitrary-width case)

Universal approximation theorem: Let $C(X, Y)$ denote the set of continuous functions from X to Y. Let $\sigma \in C(\mathbb{R}, \mathbb{R})$. Note that $(\sigma \circ x)_i = \sigma(x_i)$, so $\sigma \circ x$ denotes σ applied to each component of x.

Then σ is not polynomial if and only if for every $n \in \mathbb{N}$, $m \in \mathbb{N}$, compact $K \subseteq \mathbb{R}^n$, $f \in C(K, \mathbb{R}^m), \varepsilon > 0$ there exist $k \in \mathbb{N}$, $A \in \mathbb{R}^{k \times n}$, $b \in \mathbb{R}^k$, $C \in \mathbb{R}^{m \times k}$ such that

$$\sup_{x \in K} \|f(x) - g(x)\| < \varepsilon$$

where

$$g(x) = C \cdot (\sigma \circ (A \cdot x + b))$$

Any continuous functions defined on a compact set can be approximated arbitrarily well by a shallow neural network if the shallow neural network is arbitrarily wide.

Source:https://en.wikipedia.org/wiki/Universal_approximation_theorem

Universality of Neural Networks (Arbitrary-depth case)

Universal approximation theorem *(L1 distance, ReLU activation, arbitrary depth, minimal width).* For any Bochner–Lebesgue p-integrable function $f : \mathbb{R}^n \to \mathbb{R}^m$ and any $\epsilon > 0$, there exists a fully-connected ReLU network F of width exactly $d_m = \max\{n+1, m\}$, satisfying

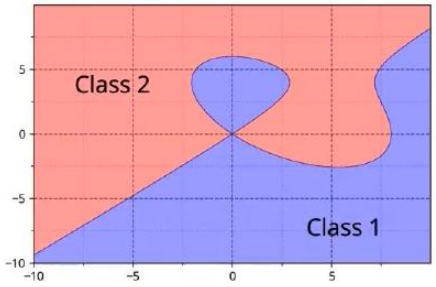
$$\int_{\mathbb{R}^n} \|f(x) - F(x)\|^p \, \mathrm{d}x < \epsilon.$$

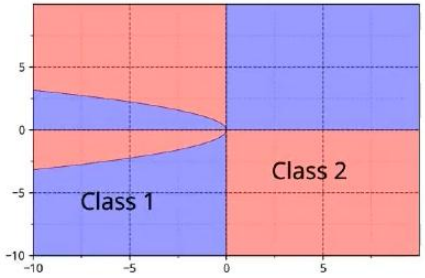
Moreover, there exists a function $f \in L^p(\mathbb{R}^n, \mathbb{R}^m)$ and some $\epsilon > 0$, for which there is no fully-connected ReLU network of width less than $d_m = \max\{n+1, m\}$ satisfying the above approximation bound.

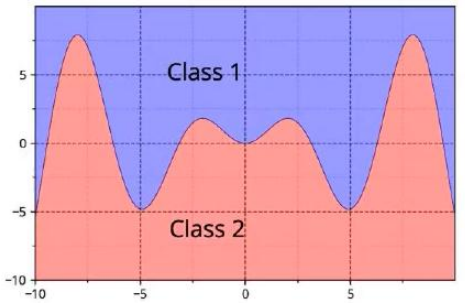
Any continuous functions defined on a compact set can be approximated arbitrarily well by a fixed-width neural network if the neural network is arbitrarily deep.

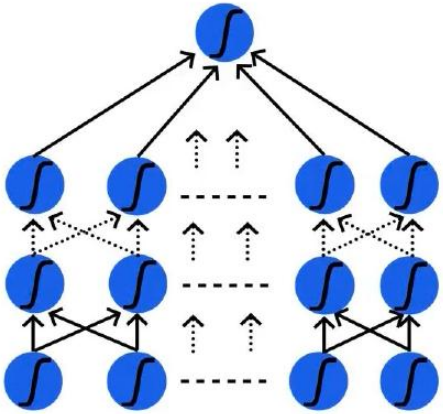
Source:https://en.wikipedia.org/wiki/Universal_approximation_theorem

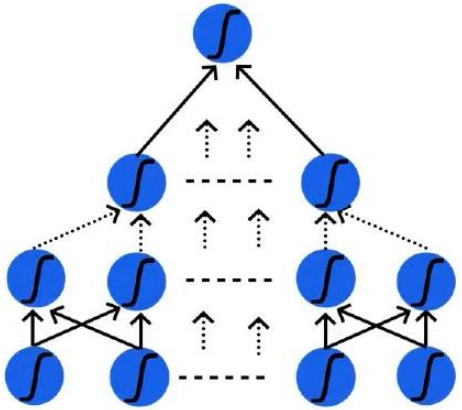
Universality of Neural Networks

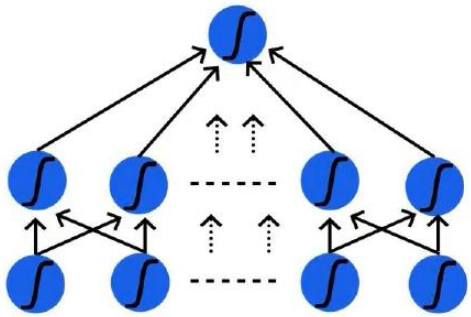












Neural networks can also approximate classification regions very well.

How to use Deep Neural Networks

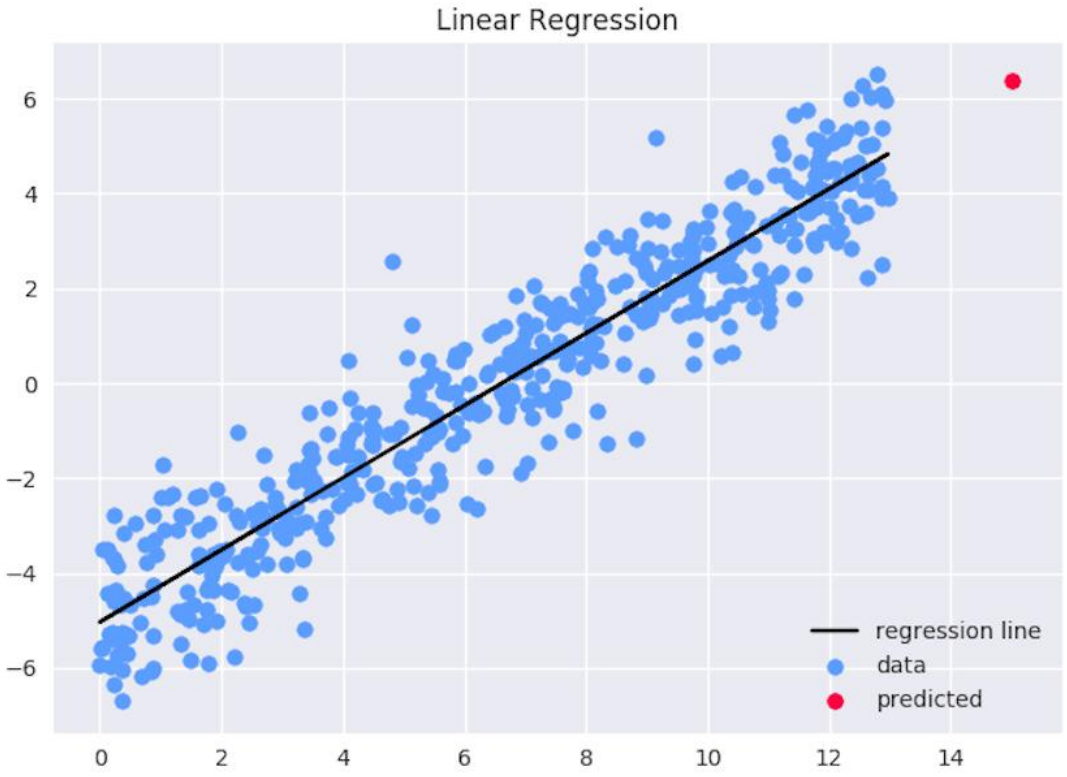
to do

regression?

Deep Neural Regressions

Deep Nonparametric Regression

An Example



- **Least squares regression**
- **Data** $(X_i, Y_i), i = 1, \dots, n.$
- **To find a**
  $f(x; \alpha, \beta) = \beta' x + \alpha$
  such that
  $\sum (Y_i - f(X_i))^2$
  is minimized over

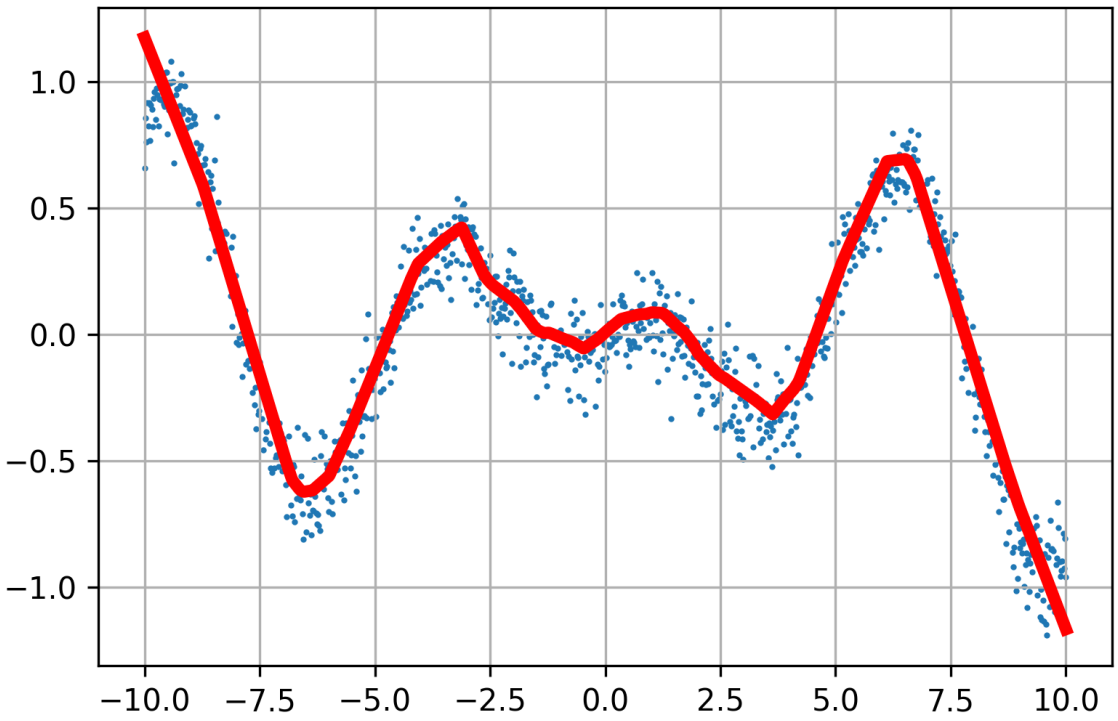
$$\mathcal{F} = \{f : f(x; \alpha, \beta) = \beta' x + \alpha, \alpha, \beta \in \mathbb{R}\}$$

- **Write** $Z_i' = (1, X_i'),\ Z' = (Z_1', Z_2', \cdots, Z_n')$, $Y' = (Y_1, Y_2, \cdots, Y_n)$ **and** $\theta' = (\alpha, \beta')$, **then the closed-form solution is**

$$\theta^* = (Z'Z)^{-1} Z'Y = argmin_{\theta \in \mathbb{R}^2} \sum \|Y - Z\theta\|^2$$

Deep Nonparametric Regression

An Example



- Data $(X_i, Y_i), i = 1, \ldots, n.$
- To find a network
  $f(x; \theta)$
  such that
  $\sum(Y_i - f(X_i; \theta))^2$
  is minimized over

$\mathcal{F} = \{f: f(x; \theta)$ *is a neural network parameterized by* $\theta \in \mathbb{R}^s\}$

- How do we solve for θ ?
  It seems hard to get no closed-form solution.
- Search for θ using optimization algorithms.
  e.g., (stochastic) gradient decent and its variants.

The optimization problem

- **Data** $(X_i, Y_i), i = 1, \dots, n$.
- **The empirical risk**

$$R_n(\theta) = R_n\big(f(\cdot, \theta)\big) = \frac{1}{n}\sum \big(Y_i - f(X_i; \theta)\big)^2 .$$

- **The target is to minimize** $R_n(\theta)$ **over** $\theta \in \mathbb{R}^s$.

The loss, objective function $R_n(\theta)$.

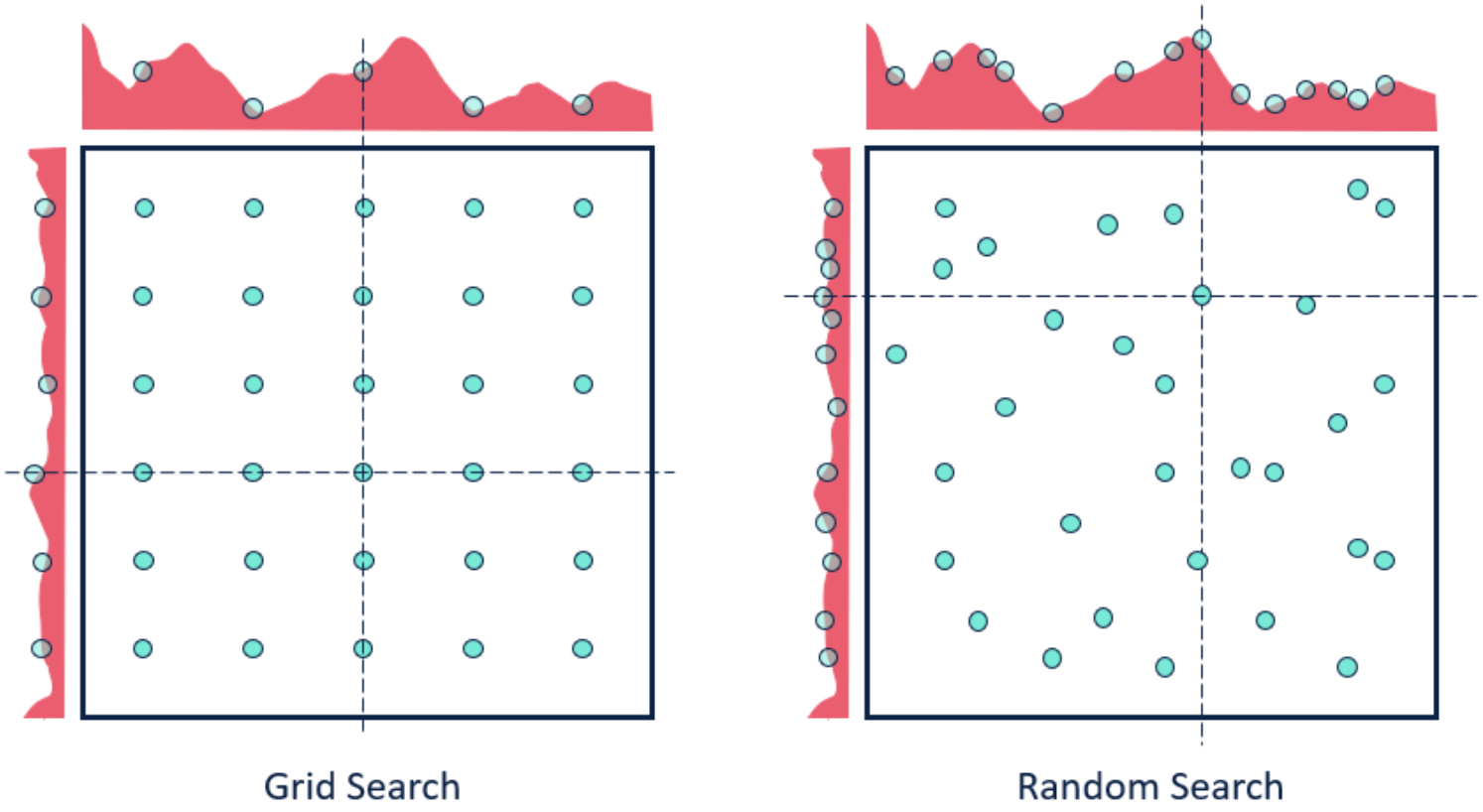
The parameter $\theta \in \mathbb{R}^s$.

The loss surface

Source: https://www.maxpixel.net/Mountains-Valleys-Landscape-Hills-Grass-Green-699369

The Optimization Problem

Strategy #1: Grid search and Random search



Source:https://community.alteryx.com/t5/Data-Science/Hyperparameter-Tuning-Black-Magic/ba-p/449289

Low efficiency, especially in high-dimensional parameter space. (Curse of dimensionality)

The Optimization Problem

Strategy #1: Random search

```
# assume X_train is the data where each column is an example (e.g. 96 x 50,000)
# assume Y_train are the responses (e.g. 1D array of 50,000)
# assume the function R evaluates the loss function
bestloss = float("inf") # Python assigns the highest possible float value
for num in range(1000):
        theta = np.random.randn(9876) * 0.0001 # generate random parameters
        loss = R(X_train, Y_train, theta) # get the loss over the entire training set
        if loss < bestloss: # keep track of the best solution
                bestloss = loss
                best_theta = theta
        print 'in attempt %d the loss was %f, best %f' % (num, loss, bestloss)
```

A toy example: code for random search.

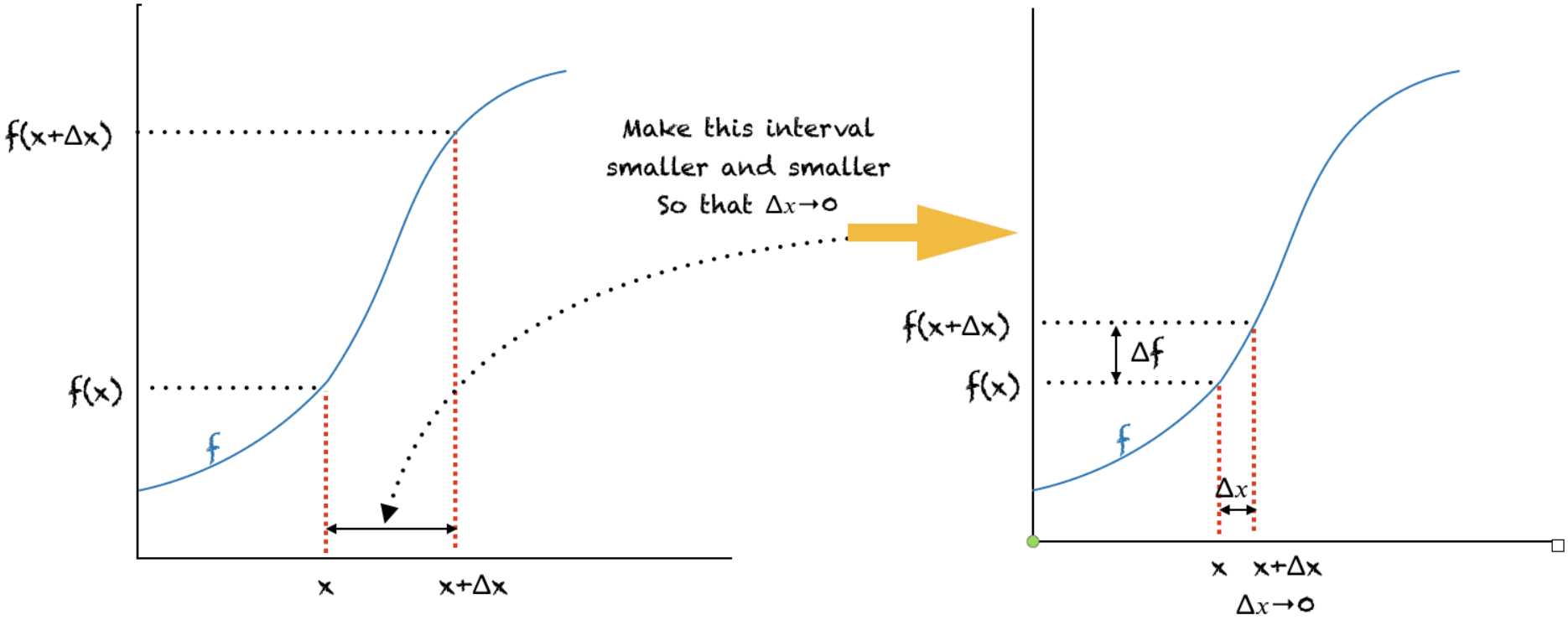
Loop: 1. Random guess 2. Check and compare 3. Update

The Optimization Problem

Strategy #2: Go along the gradient

Derivative of f is the rate of change of f

$$f'(x) = \frac{df}{dx} = \lim_{\Delta x \to 0} \frac{f(x+\Delta x) - f(x)}{\Delta x} = \lim_{\Delta x \to 0} \frac{\Delta f}{\Delta x}$$



Source:https://machinelearningmastery.com/a-gentle-introduction-to-function-derivatives/

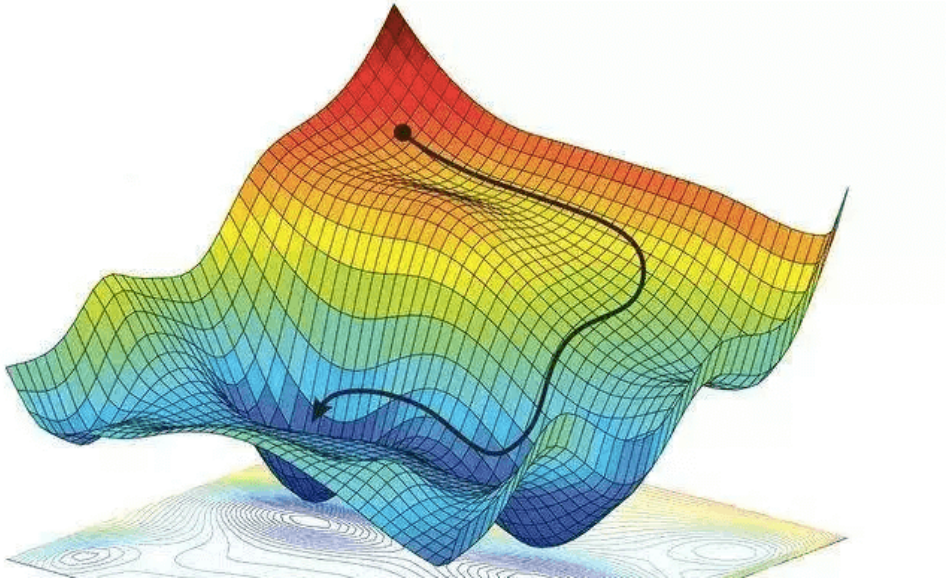
The Optimization Problem

Strategy #2: Go along the gradient

In multiple dimensions, the **gradient is the vector of (partial derivatives) along each dimension**.

The slope in any direction is the **dot product of the direction with the gradient.**

$$\nabla f = \begin{bmatrix} \frac{\partial f}{\partial x} \\ \\ \frac{\partial f}{\partial y} \\ \\ \vdots \end{bmatrix}$$



The direction of **steepest descent** is the **negative gradient.**

The Optimization Problem

Strategy #2: Go along the gradient. (Example)

Let W denotes the parameter to be optimized.

current W:

[0.34,
-1.11,
0.78,
0.12,
0.55,
2.81,
-3.1,
-1.5,
0.33,…]

loss 1.25347

gradient dW:

[?,
?,
?,
?,
?,
?,
?,
?,
?,…]

Source: http://cs231n.stanford.edu/slides/2022/lecture_3_ruohan.pdf

The Optimization Problem

Strategy #2: Go along the gradient. (Example)

See how does loss change along the first dimension.

current W:	**W + h** (first dim)**:**	**gradient dW:**
[0.34, -1.11, 0.78, 0.12, 0.55, 2.81, -3.1, -1.5, 0.33,…]	[0.34 + **0.0001**, -1.11, 0.78, 0.12, 0.55, 2.81, -3.1, -1.5, 0.33,…]	[?, ?, ?, ?, ?, ?, ?, ?, ?,…]
loss 1.25347	**loss 1.25322**	

Source: http://cs231n.stanford.edu/slides/2022/lecture_3_ruohan.pdf

Strategy #2: Go along the gradient. (Example)

Calculate the **partial derivative** along the first dimension.

current W:	**W + h** (first dim)**:**	**gradient dW:**
[0.34, -1.11, 0.78, 0.12, 0.55, 2.81, -3.1, -1.5, 0.33,…]	[0.34 + **0.0001**, -1.11, 0.78, 0.12, 0.55, 2.81, -3.1, -1.5, 0.33,…]	[**-2.5**, ?, ?, ?, ?,…]
loss 1.25347	**loss 1.25322**	

(1.25322 - 1.25347)/0.0001
= -2.5

$$\frac{df(x)}{dx} = \lim_{h \to 0} \frac{f(x+h) - f(x)}{h}$$

Source: http://cs231n.stanford.edu/slides/2022/lecture_3_ruohan.pdf

The Optimization Problem

Strategy #2: Go along the gradient. (Example)

See how does loss change along the second dimension.

current W:	**W + h** (second dim)**:**	**gradient dW:**
[0.34,	[0.34,	[-2.5,
-1.11,	-1.11 + **0.0001**,	?,
0.78,	0.78,	?,
0.12,	0.12,	?,
0.55,	0.55,	?,
2.81,	2.81,	?,
-3.1,	-3.1,	?,
-1.5,	-1.5,	?,
0.33,…]	0.33,…]	?,…]
loss 1.25347	**loss 1.25353**	

Source: http://cs231n.stanford.edu/slides/2022/lecture_3_ruohan.pdf

The Optimization Problem

Strategy #2: Go along the gradient. (Example)

Calculate the **partial derivative** along the second dimension.

current W:	**W + h** (second dim):	**gradient dW:**
[0.34,	[0.34,	[-2.5,
-1.11,	-1.11 + **0.0001**,	**0.6**,
0.78,	0.78,	?,
0.12,	0.12,	?,
0.55,	0.55,	
2.81,	2.81,	
-3.1,	-3.1,	
-1.5,	-1.5,	
0.33,…]	0.33,…]	?,…]
loss 1.25347	**loss 1.25353**	

(1.25353 - 1.25347)/0.0001
= 0.6

$$\frac{df(x)}{dx} = \lim_{h \to 0} \frac{f(x+h) - f(x)}{h}$$

Source: http://cs231n.stanford.edu/slides/2022/lecture_3_ruohan.pdf

The Optimization Problem

Strategy #2: Go along the gradient. (Example)

See how does loss change along the third dimension.

current W:	**W + h** (third dim)**:**	**gradient dW:**
[0.34, -1.11, 0.78, 0.12, 0.55, 2.81, -3.1, -1.5, 0.33,…]	[0.34, -1.11, 0.78 + **0.0001**, 0.12, 0.55, 2.81, -3.1, -1.5, 0.33,…]	[-2.5, 0.6, ?, ?, ?, ?, ?, ?, ?,…]
loss 1.25347	**loss 1.25347**	

Source: http://cs231n.stanford.edu/slides/2022/lecture_3_ruohan.pdf

The Optimization Problem

Strategy #2: Go along the gradient. (Example)

Calculate the **partial derivative** along the third dimension.

current W:	**W + h** (third dim):	**gradient dW:**
[0.34,	[0.34,	[-2.5,
-1.11,	-1.11,	0.6,
0.78,	0.78 + **0.0001**,	**0**,
0.12,	0.12,	?,
0.55,	0.55,	
2.81,	2.81,	
-3.1,	-3.1,	
-1.5,	-1.5,	
0.33,…]	0.33,…]	?,…]
loss 1.25347	**loss 1.25347**	

(1.25347 - 1.25347)/0.0001
= 0

$$\frac{df(x)}{dx} = \lim_{h \to 0} \frac{f(x+h) - f(x)}{h}$$

Source: http://cs231n.stanford.edu/slides/2022/lecture_3_ruohan.pdf

The Optimization Problem

Strategy #2: Go along the gradient. (Example)

Computing numerical gradient is slow! We need to do too much steps.

current W:	**W + h** (third dim)**:**	**gradient dW:**
[0.34,	[0.34,	[-2.5,
-1.11,	-1.11,	0.6,
0.78,	0.78 + **0.0001**,	**0**,
0.12,	0.12,	?,
0.55,	0.55,	
2.81,	2.81,	
-3.1,	-3.1,	
-1.5,	-1.5,	
0.33,…]	0.33,…]	?,…]
loss 1.25347	**loss 1.25347**	

Numeric Gradient
- Slow! Need to loop over all dimensions
- Approximate

Source: http://cs231n.stanford.edu/slides/2022/lecture_3_ruohan.pdf

The Optimization Problem

A fast way to find gradient: use Calculus to compute an analytic gradient.

current W:

[0.34,
-1.11,
0.78,
0.12,
0.55,
2.81,
-3.1,
-1.5,
0.33,…]

loss 1.25347

dW = ...
(some function
data and W)

gradient dW:

[-2.5,
0.6,
0,
0.2,
0.7,
-0.5,
1.1,
1.3,
-2.1,…]

The Optimization Problem

A fast way to find gradient: use Calculus to compute an analytic gradient.

Newton and Leibniz

Recall: least squares empirical risk $R_n(\theta) = \frac{1}{n}\sum\left(Y_i - f(X_i;\theta)\right)^2$

Gradient: $\frac{d}{d\theta}R_n(\theta) = -\frac{2}{n}\sum(Y_i - f(X_i;\theta))\frac{d}{d\theta}f(X_i;\theta)$

To compute $\frac{d}{d\theta}f(X_i;\theta)$ **, use Chain rule.**

Gradient Decent

- **Numerical Gradient**: approximate, slow, easy to write
- **Analytic Gradient:** exact, fast, error-prone

In practice: use analytic gradient, but check implementation with numerical gradient (gradient check).

```
# Vanilla Gradient Descent
while True:
        parameters_grad = evaluate_gradient(loss_fun, data, parameters)
        parameters += - step_size * parameters_grad  # perform parameter update
```

The Optimization Problem

The optimization problem

- Data $(X_i, Y_i), i = 1, \dots, n.$
- The empirical risk

$$R_n(\theta) = R_n\big(f(\cdot, \theta)\big) = \frac{1}{n}\sum \big(Y_i - f(X_i; \theta)\big)^2 .$$

- To minimize $R_n(\theta)$ over $\theta \in \mathbb{R}^s$.

Initialize $\boldsymbol{\theta_0} \in \mathbb{R}^s$ by some randomization

For $\boldsymbol{t = 1, \cdots, T}$

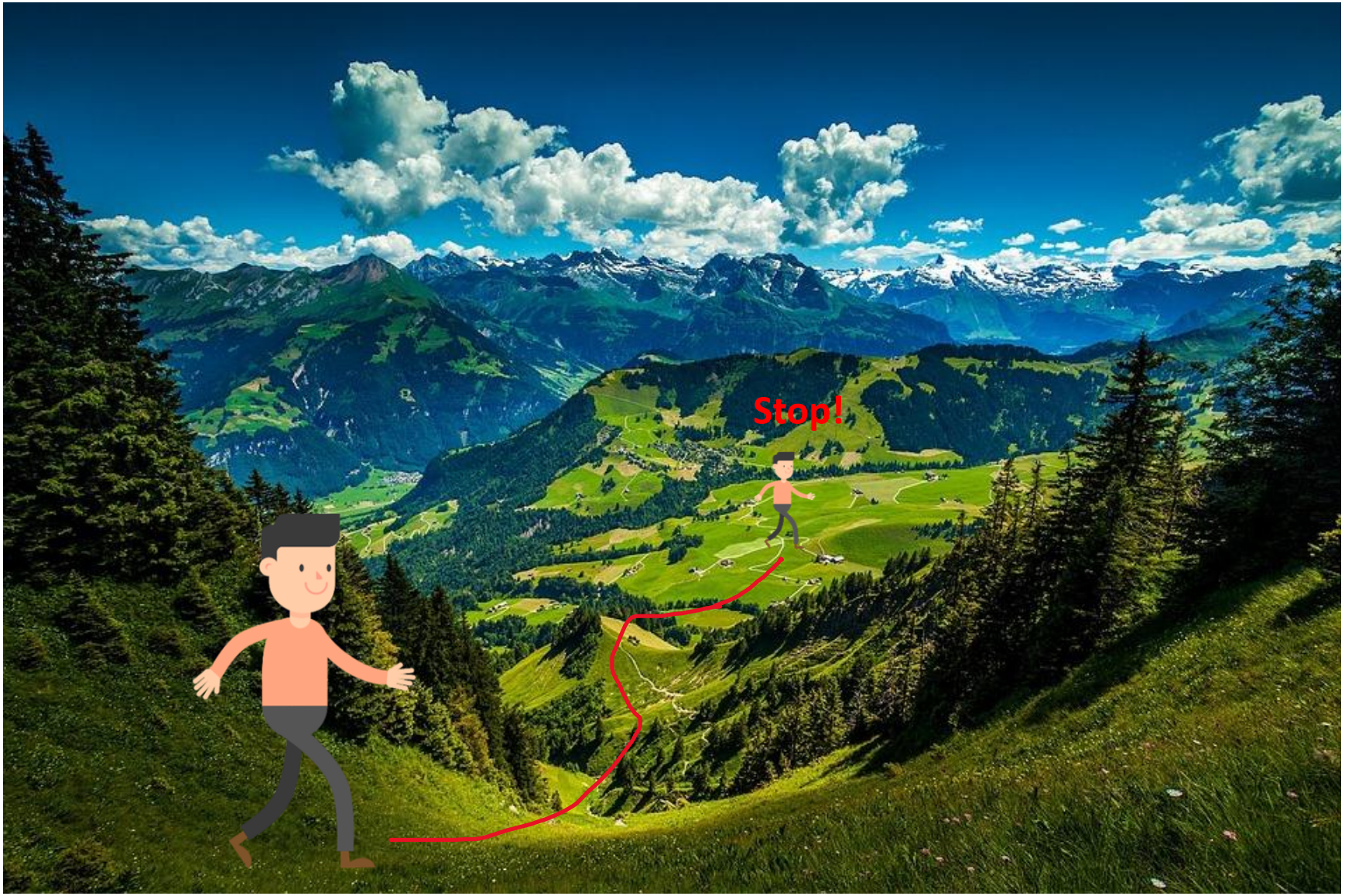
Calculate $\frac{\boldsymbol{dR_n(\theta)}}{\boldsymbol{d\theta}}\big|_{\boldsymbol{\theta=\theta_{t-1}}}$

Set stepsize $\boldsymbol{\alpha_t > 0}$

Update $\boldsymbol{\theta_t = \theta_{t-1} - \alpha_t \cdot [\frac{dR_n(\theta)}{d\theta}\big|_{\theta=\theta_{t-1}}]}$

After $\boldsymbol{T}$ times iterations, we got $\boldsymbol{\theta_T}$ such that $\boldsymbol{R_n(\theta_T)}$ is small.

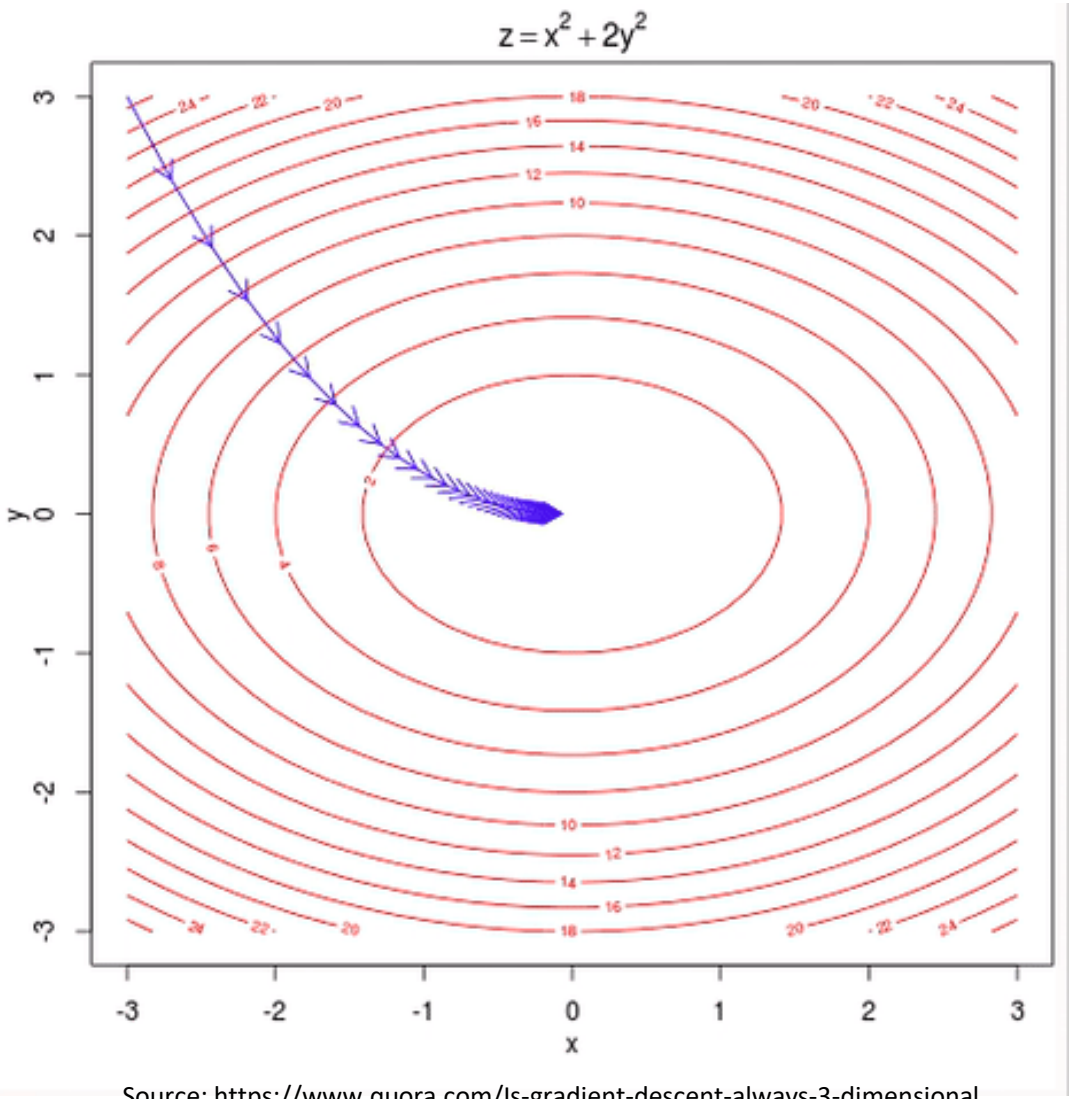
The loss surface



Stop somewhere after some iterations of optimization.

Gradient decent

An example of gradient decent



Source: https://www.quora.com/Is-gradient-descent-always-3-dimensional

The training process

The optimization problem

- Data $(X_i, Y_i), i = 1, \dots, n.$
- The empirical risk

$$R_n(\theta) = R_n\big(f(\cdot, \theta)\big) = \frac{1}{n}\sum \big(Y_i - f(X_i; \theta)\big)^2 .$$

- To minimize $R_n(\theta)$ over $\theta \in \mathbb{R}^s$.

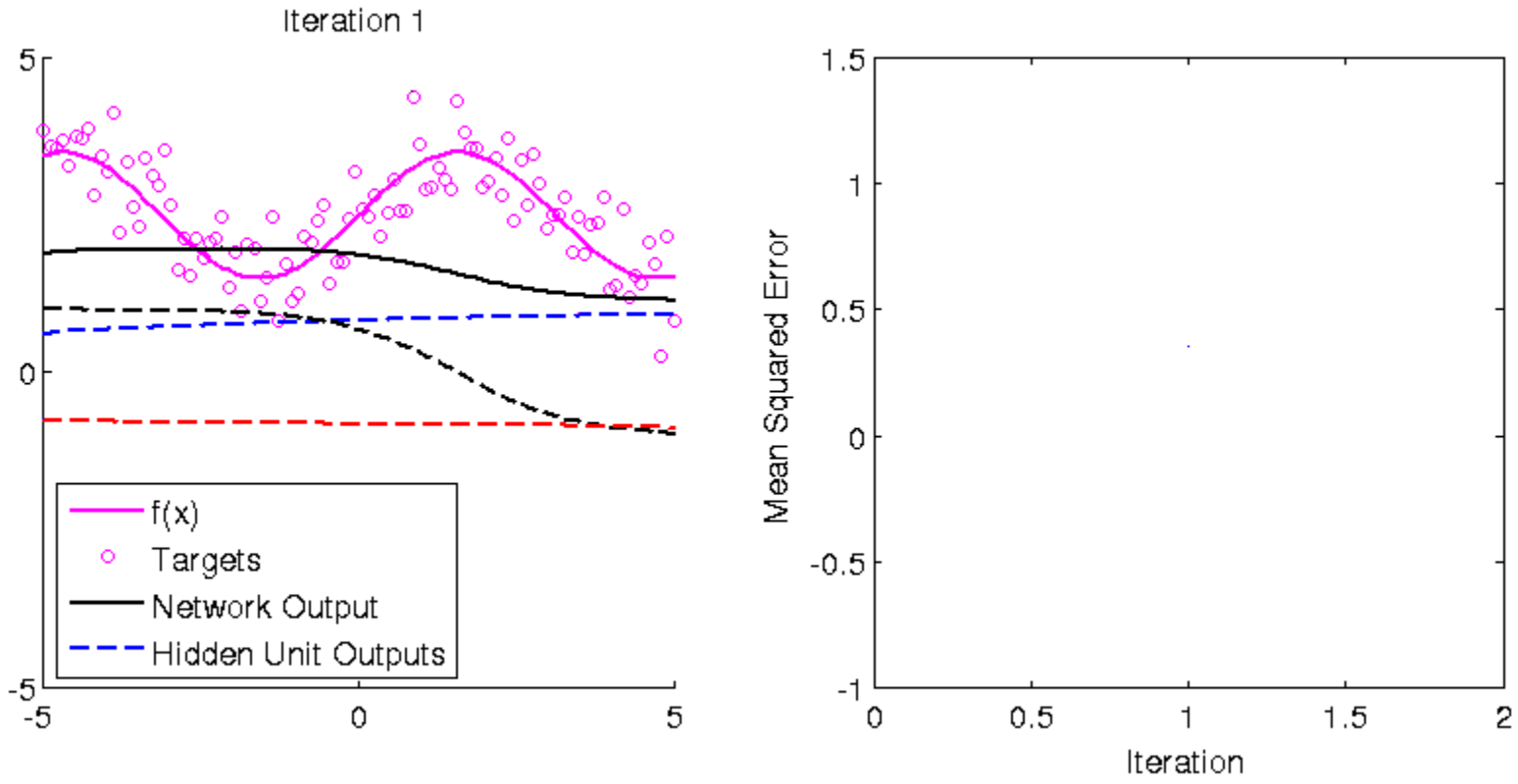
Let's see how the risk $R_n(\theta_t)$

and the neural network prediction $f(\cdot\,; \theta_t)$

change along the iterations.

The training process

An example of deep nonparametric regression



Source: https://i.gifer.com/JoRy.gif

The Optimization Problem

Questions

1. How to initialize $\theta_0 \in \mathbb{R}^s$?

2. How to choose the stepsize $\alpha_t > 0$?

3. How to calculate the gradient

$$\frac{d}{d\boldsymbol{\theta}} \boldsymbol{R_n}(\boldsymbol{\theta}) = -\frac{2}{n}\sum(\boldsymbol{Y_i} - \boldsymbol{f}(\boldsymbol{X_i}; \boldsymbol{\theta}))\frac{d}{d\boldsymbol{\theta}}\boldsymbol{f}(\boldsymbol{X_i}; \boldsymbol{\theta})$$

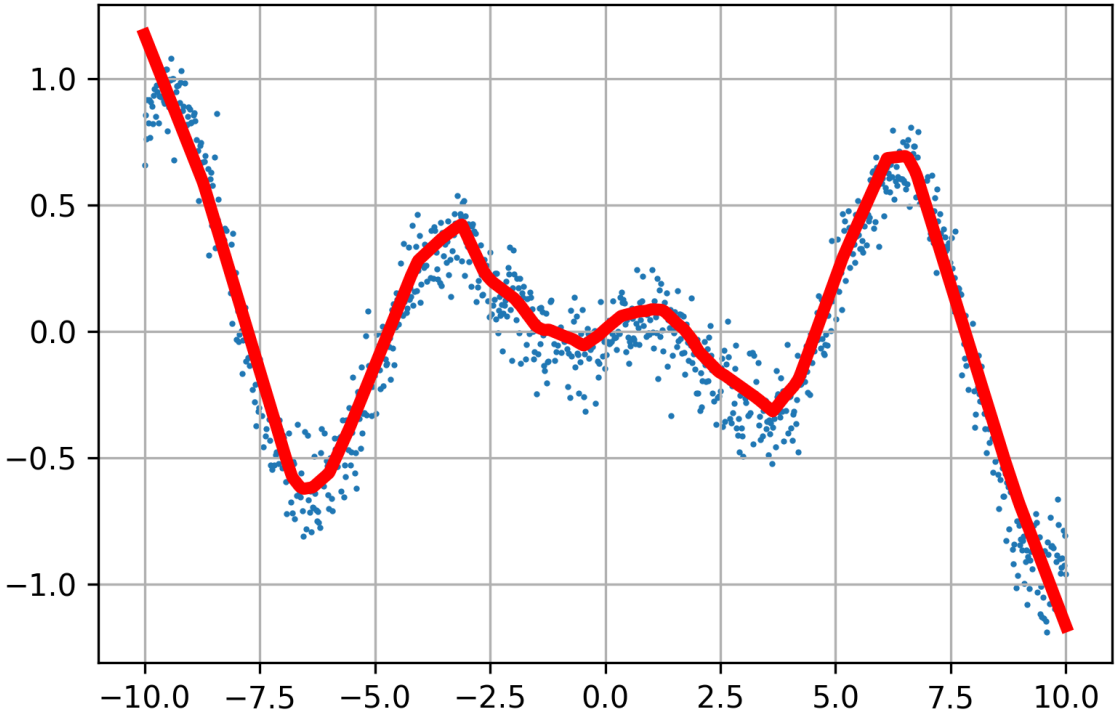
especially how to compute $\frac{d}{d\boldsymbol{\theta}}\boldsymbol{f}(\boldsymbol{X_i}; \boldsymbol{\theta})$ exactly?

4. What if the sample size $\boldsymbol{n}$ is very large?

Optimization techniques. Talk about it later.

Deep Nonparametric Regression

Recall the regression problem



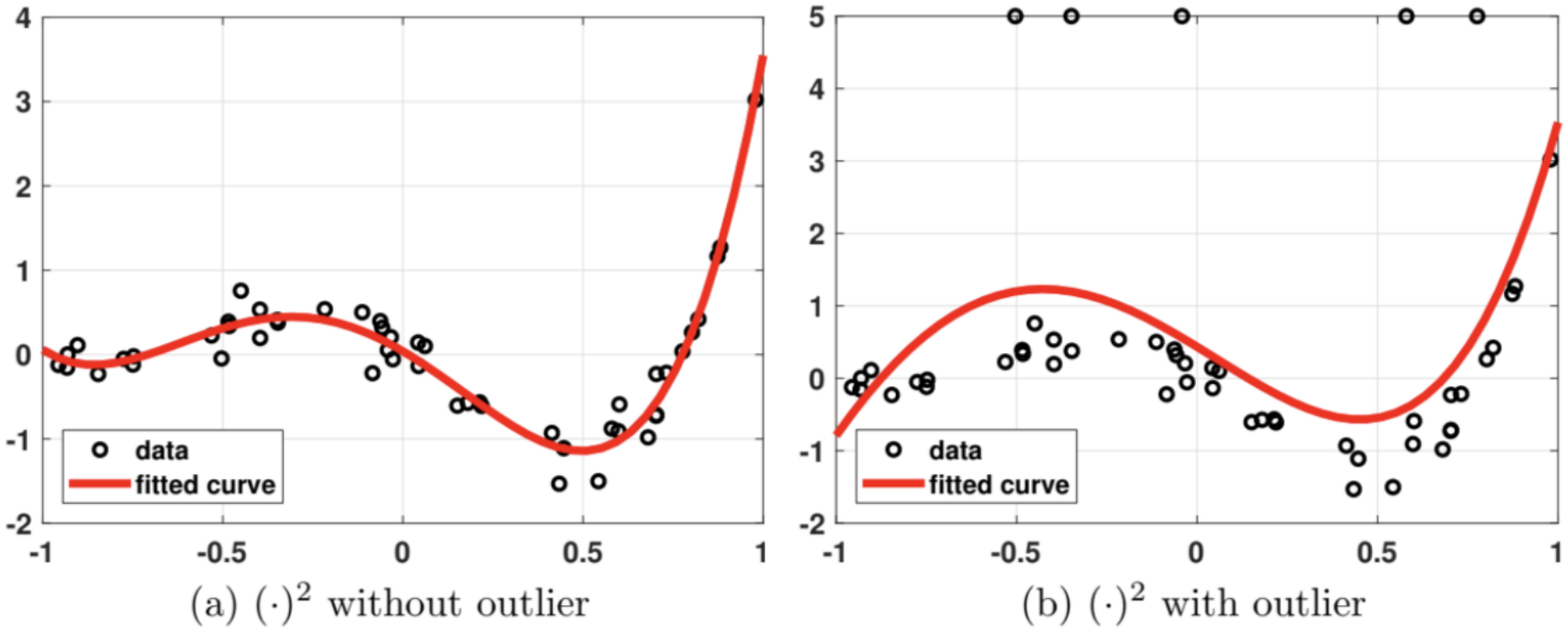
- Data $(X_i, Y_i), i = 1, \ldots, n.$
- To find a network
  $f(x; \theta)$
  such that
  $\sum (Y_i - f(X_i; \theta))^2$
  is minimized over

$\mathcal{F} = \{f : f(x; \theta)$ *is a neural network parameterized by* $\theta \in \mathbb{R}^s\}$

- How do we solve for θ ?
  1. Initialize $\theta_0 \in \mathbb{R}^s$
  2. Calculate the gradient at θ_t
  3. Move a step α_t
  4. Iterate until stop.

Deep Neural Regressions with General loss

Problem: Regression with outliers

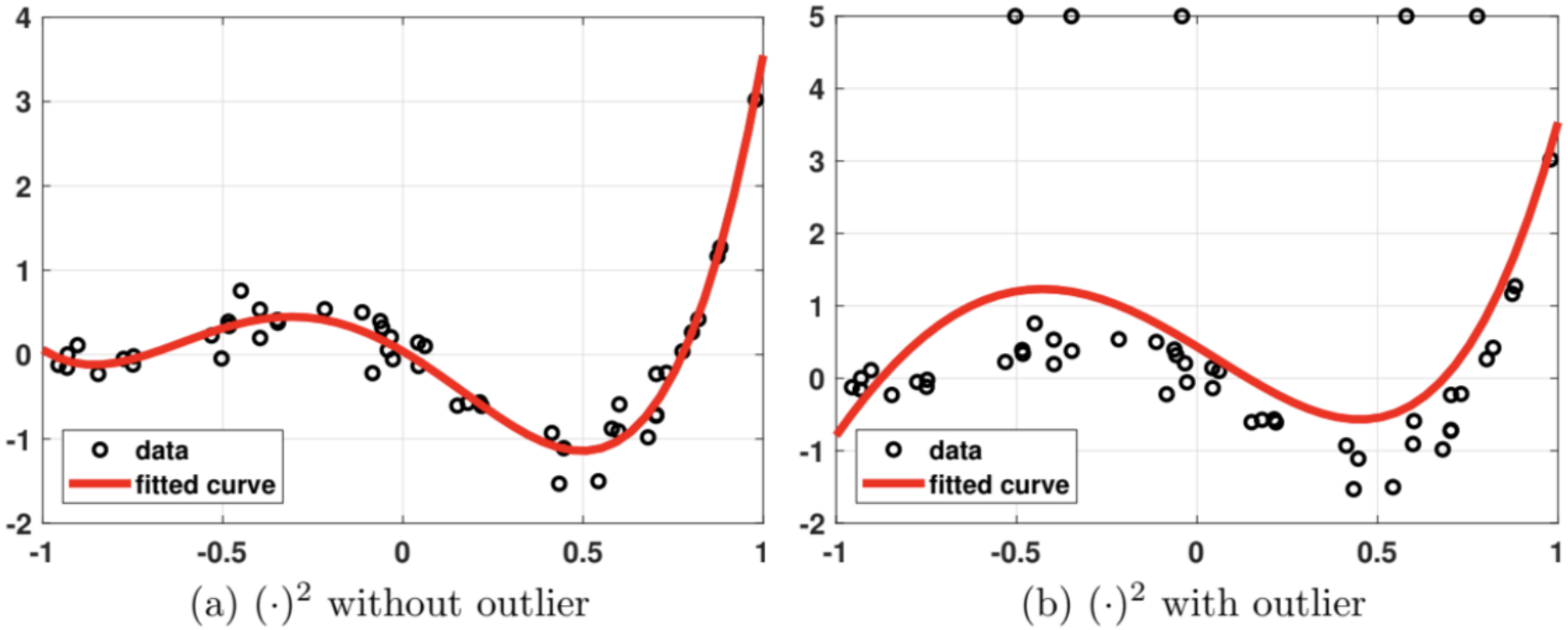


Source: Figure 7.11 of the book "Intro to Probability for Data Science"

Contaminated data can deteriorate the least squares regression.

Deep Nonparametric Regression

Problem: Regression with outliers



Source: Figure 7.11 of the book "Intro to Probability for Data Science"

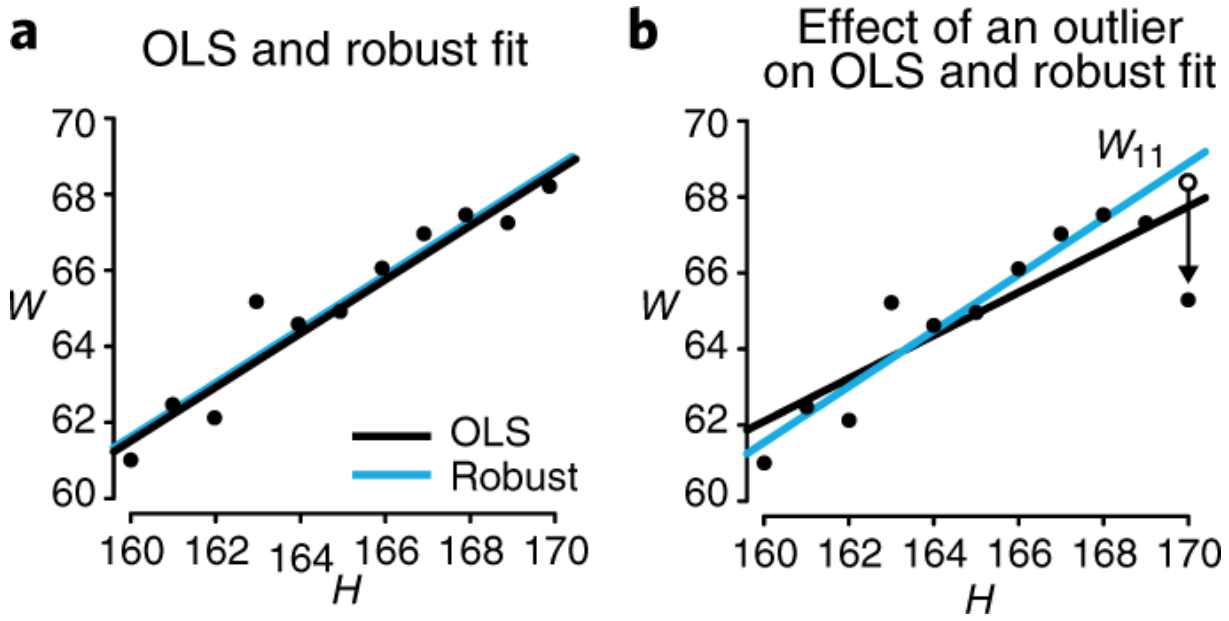
But Why do least square regression fail?

Deep Nonparametric Regression

Problem: Regression with outliers

- Data $(X_i, Y_i), i = 1, \dots, n$. Minimize empirical risk $R_n(\theta) = R_n\big(f(\cdot, \theta)\big) = \frac{1}{n}\sum\big(Y_i - f(X_i; \theta)\big)^2$ over $\theta \in \mathbb{R}^s$. Or minimize $R_n(f)$ over $\mathcal{F}$.
- Let $R(f) = \mathbb{E}\left(Y_i - f(X_i)\right)^2$ be the risk of a function f, then for each x
  $f^*(x) = \mathbb{E}[Y|X = x] = \text{argmin}_f\ \mathbb{E}\left[\left(Y_i - f(X_i)\right)^2 \middle| X_i = x\right]$. (Try proving it.)

- Then the targets of the minimization of $R_n(f)$ is the conditional mean of Y given X. In other words, least squares regression targets for "conditional mean".

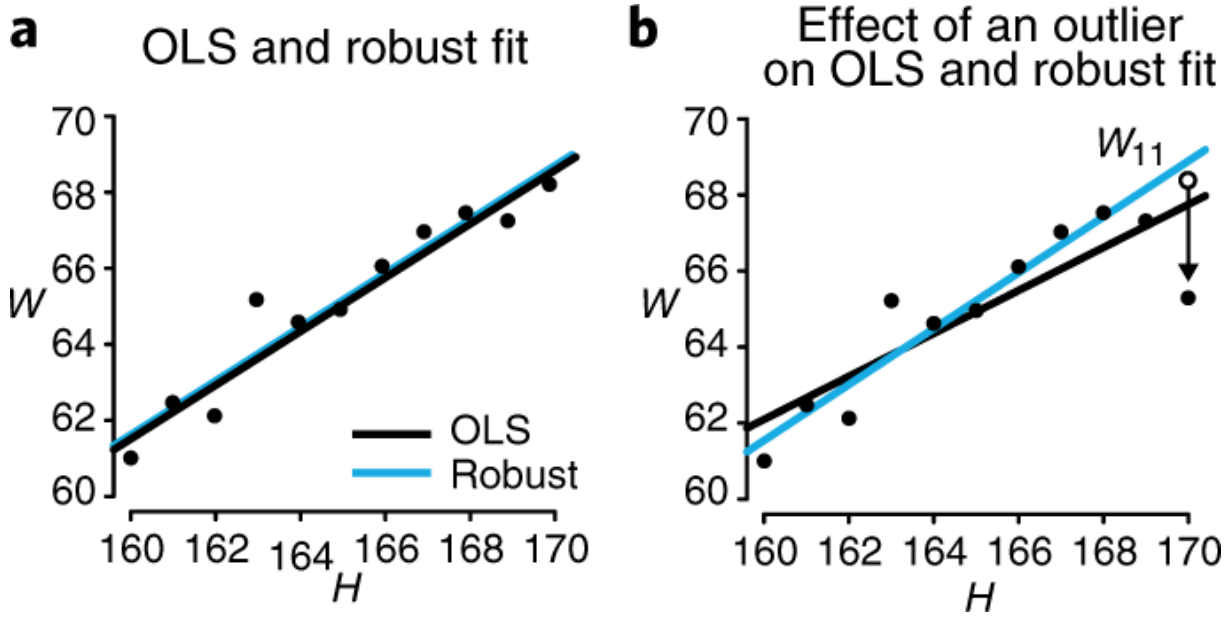


Source:https://www.nature.com/articles/s41592-019-0369-z

Deep Nonparametric Regression

Problem: Regression with outliers

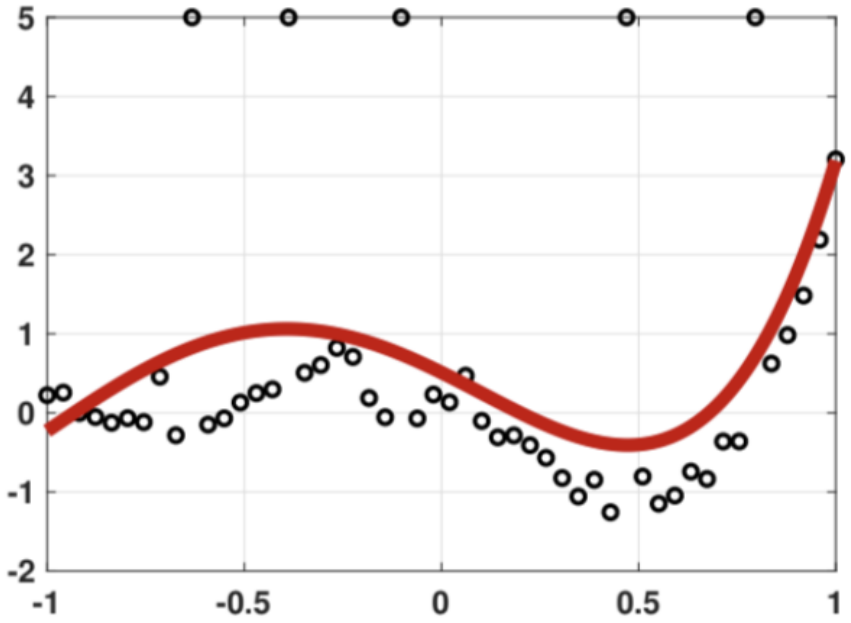
- Data $(X_i, Y_i), i = 1, \dots, n$. Minimize empirical risk $R_n(\theta) = R_n\big(f(\cdot, \theta)\big) = \frac{1}{n}\sum\big(Y_i - f(X_i; \theta)\big)^2$ over $\theta \in \mathbb{R}^s$
- Let Y_j be the contaminated data, $|Y_j - f\big(X_j; \theta\big)|$ can be large, and $\Big(Y_j - \hat{f}\big(X_j; \theta\big)\Big)^2$ is even larger such that it dominates the empirical risk $R_n(\theta) = \frac{1}{n}\sum\big(Y_i - f(X_i; \theta)\big)^2$.
- Then the minimizer $\hat{f}(\cdot, \theta)$ of $R_n(\theta)$ will be prone to fit $\big(X_j, Y_j\big)$ with priority.



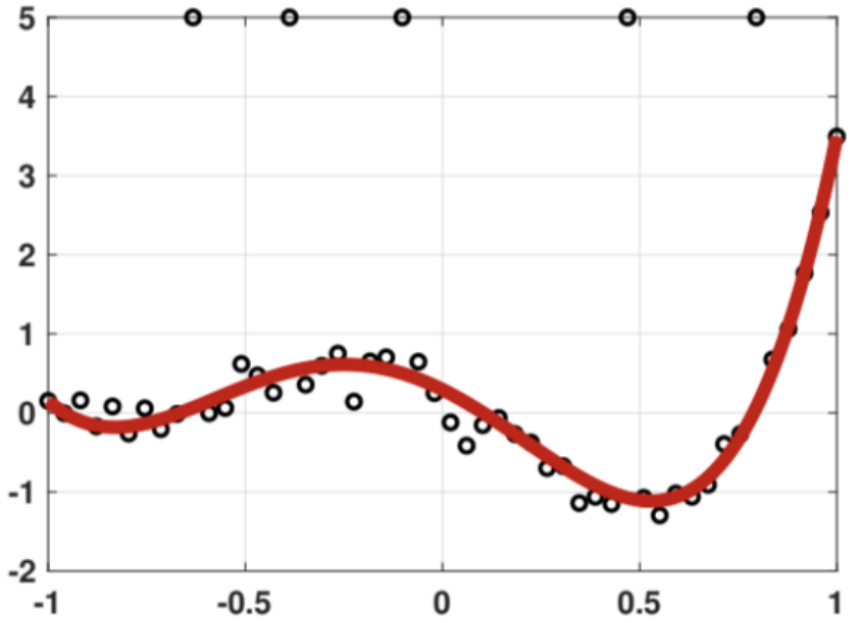
Source:https://www.nature.com/articles/s41592-019-0369-z

Deep Nonparametric Regression

Problem: Regression with outliers



(a) Ordinary $(\cdot)^2$ regression with outliers



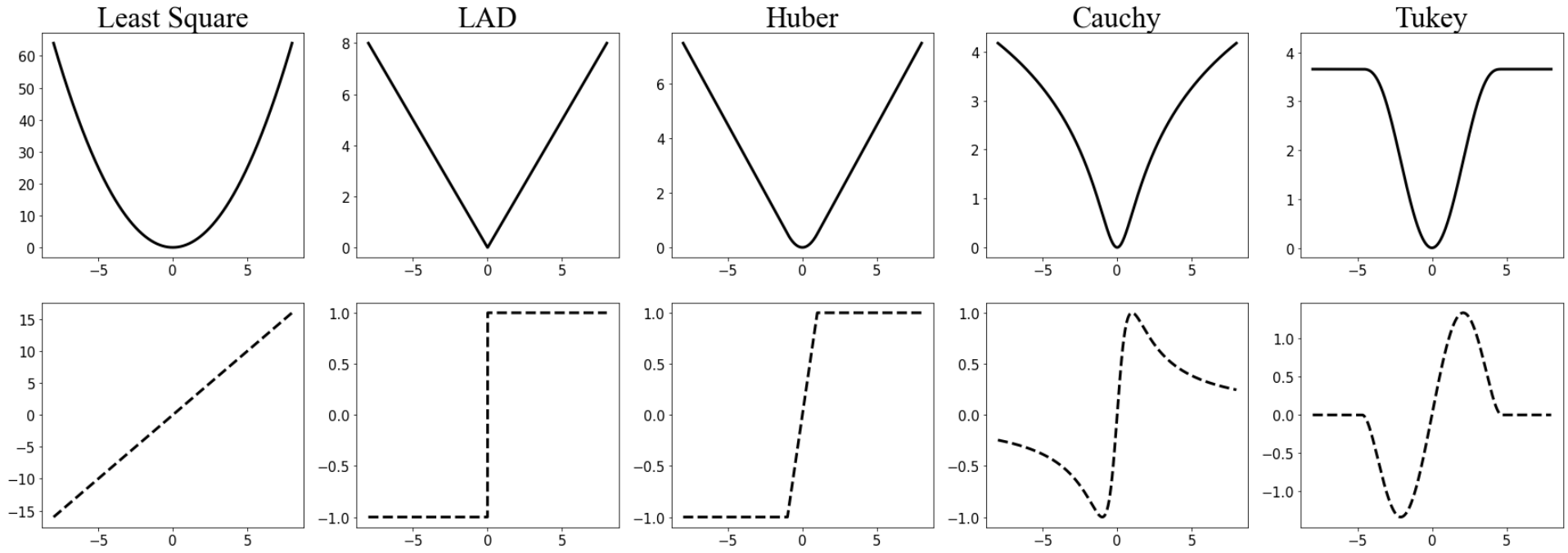
(b) Robust $|\cdot|$ regression with outliers

Source: Figure 7.12 of the book "Intro to Probability for Data Science"

Put less weight on each individual data point.

Use robust loss function.

Loss functions and their derivatives



Robust loss functions try to downplay the importance of the data point with large deviation.

Deep Nonparametric Regression

The regression problem

- **Data** $(X_i, Y_i), i = 1, \ldots, n.$
- **To find a network** $f(x; \theta)$ **such that**

$$\frac{1}{n}\sum L(Y_i, f(X_i; \theta))$$

is minimized over

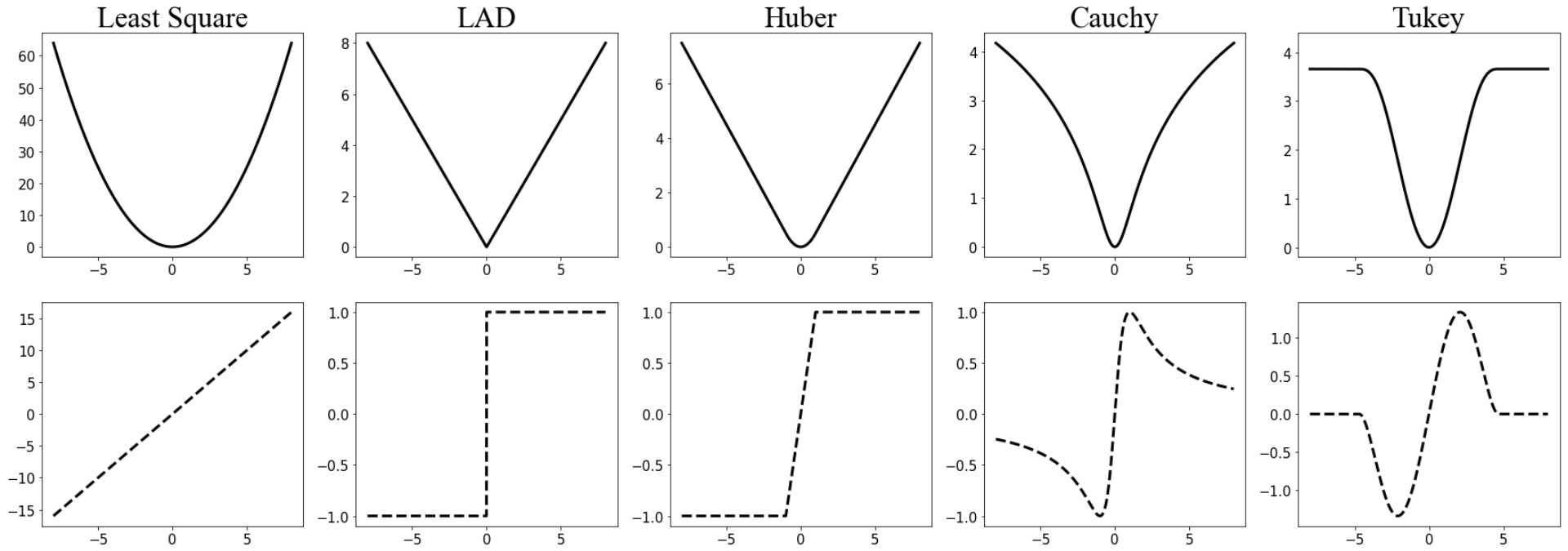
$$\mathcal{F} = \{f : f(x; \theta) \text{ is a neural network parameterized by } \theta \in \mathbb{R}^s\}.$$

- $L(\cdot,\cdot)$: $\mathbb{Y} \times \mathbb{Y} \to \mathbb{R}^+$ **is a loss function.**
- **In particular, consider** $L(a, b) = \phi(a - b)$ **for some (symmetric)** ϕ**.**

$$\frac{1}{n}\sum \phi(Y_i - f(X_i; \theta))$$

Deep Nonparametric Regression

Loss functions and their derivatives



Least square (LS): $\phi(a) = a^2$

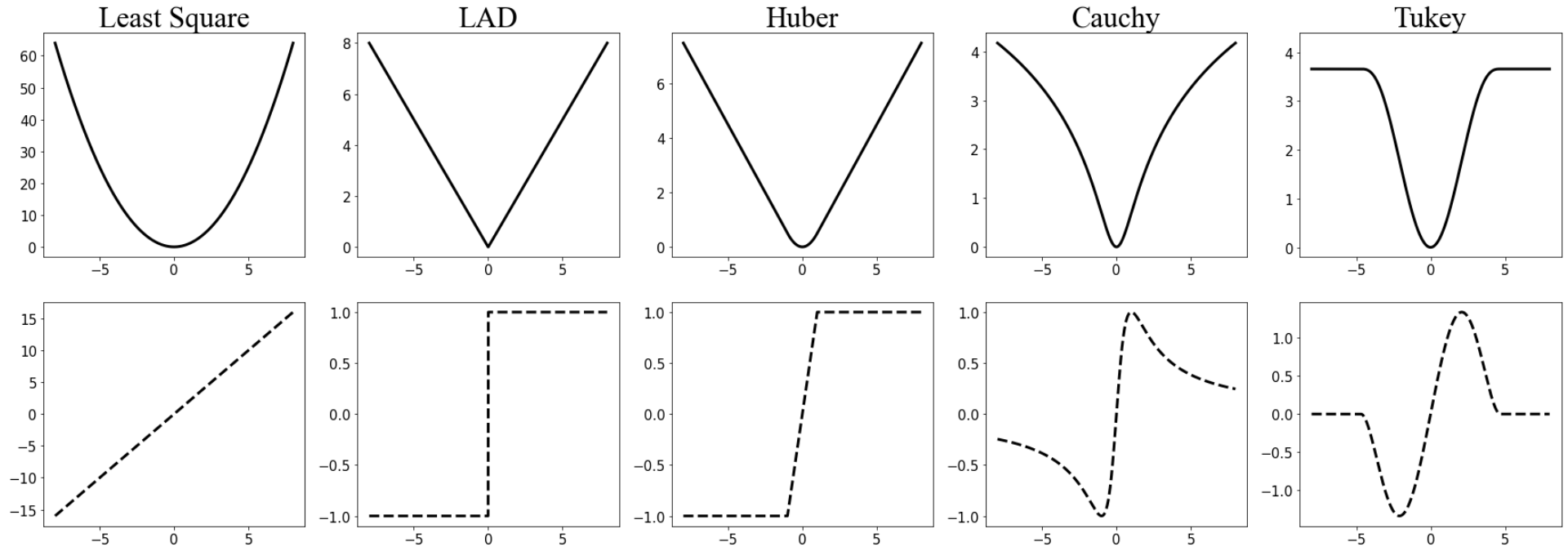
Least absolute deviation (LAD): $\phi(a) = |a|$

Huber loss: $\phi(a) = \frac{a^2}{2} \; if \; |a| < \tau; \; \phi(a) = \tau|a| - \frac{\tau^2}{2} \; if \; |a| \geq \tau$ **for some** $\tau > 0$

Cauchy loss: $\phi(a) = log\,[1 + \kappa^2 a^2]$ **for some** $\kappa > 0$

Tukey loss: $\phi(a) = \frac{t^2\left\{1-\left[1-\left(\frac{a}{t}\right)^2\right]^3\right\}}{6} \; if \; |a| < t; \phi(a) = t^2/6 \; otherwise$

Loss functions and their derivatives



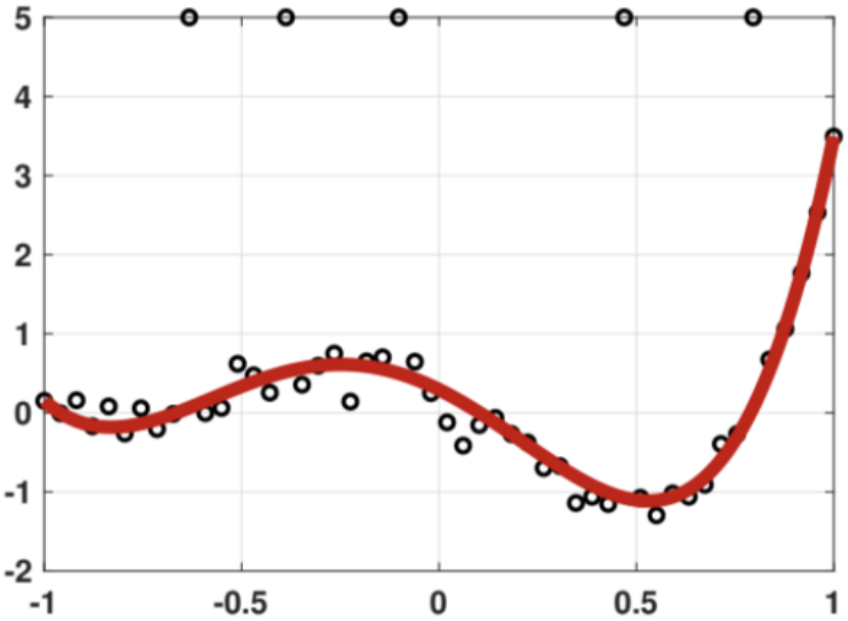
Recall: the empirical risk $R_n(\theta) = \frac{1}{n}\sum \phi(Y_i - f(X_i;\theta))$

Gradient: $\frac{d}{d\theta} R_n(\theta) = -\frac{1}{n}\sum \phi'(Y_i - f(X_i;\theta)) \frac{d}{d\theta} f(X_i;\theta)$

To compute $\frac{d}{d\theta} f(X_i;\theta)$ **, use Chain rule.**

Deep Nonparametric Regression

Recall the regression problem



- Data $(X_i, Y_i), i = 1, \ldots, n.$
- To find a network
  $f(x; \theta)$
  such that
  $\sum \phi(Y_i - f(X_i; \theta))$
  is minimized over

$\mathcal{F} = \{f: f(x; \theta)$ *is a neural network parameterized by* $\theta \in \mathbb{R}^s\}$

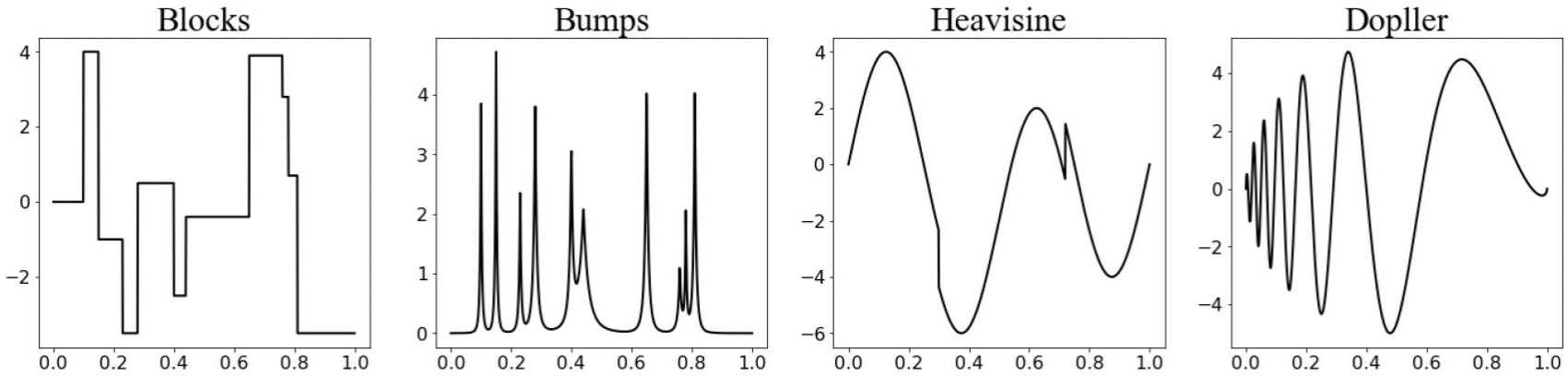
- How do we solve for θ ?
  1. Initialize $\theta_0 \in \mathbb{R}^s$
  2. Calculate the gradient at θ_t (with different ϕ)
  3. Move a step α_t
  4. Iterate until stop

Deep Nonparametric Regression

Numerical examples

Data generated from the model $Y = f_0(X) + \eta$ with sample size $n = 128$

Four target functions $f_0(X)$ are considered:



Source: https://arxiv.org/abs/2107.10343

Two types of error η are considered:

1. Standard Cauchy distribution(with location parameter 0 and scale parameter 1), denoted by $\eta \sim \text{Cauchy}(0; 1)$;

2. Normal mixture distribution, denoted by "Mixture", where $\eta \sim 0.8 \times N(0,1) + 0.2 \times N(0, 10^4)$;

Setup for training

Loss functions

Least square (LS): $\phi(a) = a^2$

Least absolute deviation (LAD): $\phi(a) = |a|$

Huber loss: $\phi(a) = \frac{a^2}{2} if\ |a| < \tau;\ \phi(a) = \tau|a| - \frac{\tau^2}{2}\ if\ |a| \geq \tau$ for $\tau = 1.345$

Cauchy loss: $\phi(a) = log\ [1 + \kappa^2 a^2]$ for $\kappa = 1$

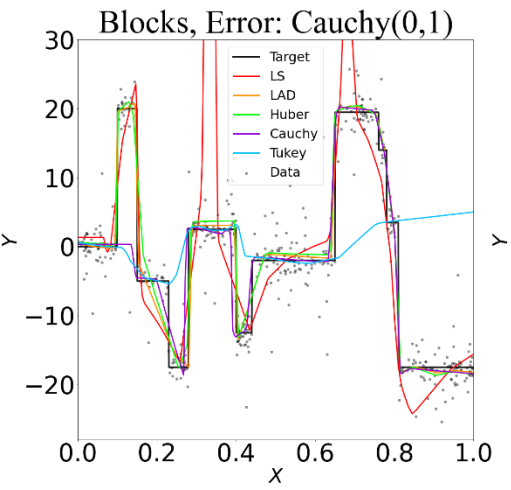
Tukey loss: $\phi(a) = \frac{t^2\left\{1-\left[1-\left(\frac{a}{t}\right)^2\right]^3\right\}}{6}\ if\ |a| < t; \phi(a) = \frac{t^2}{6} otherwise$ for $t = 4.685$

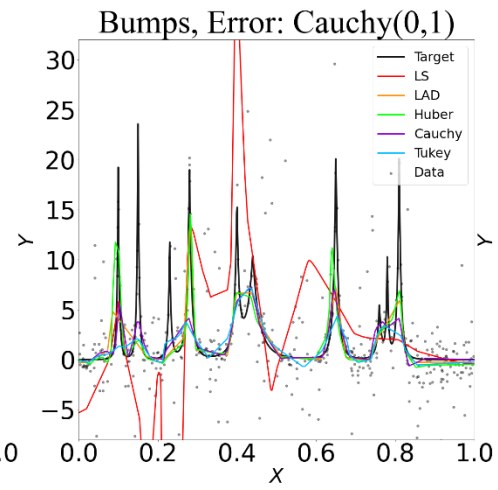
Neural Network

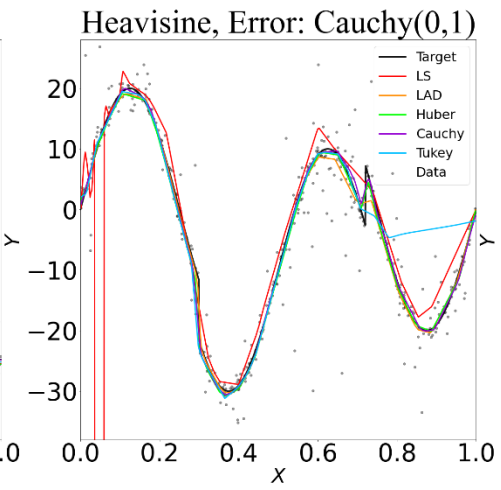
ReLU activated multilayer perceptrons with 5 hidden layers and network width being (1; 256; 256; 256; 256; 256; 1).

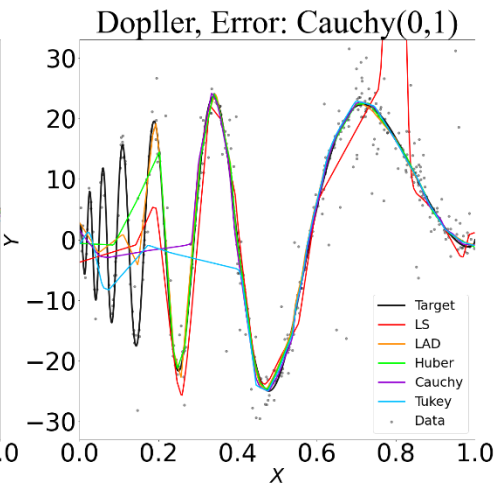
Deep Nonparametric Regression

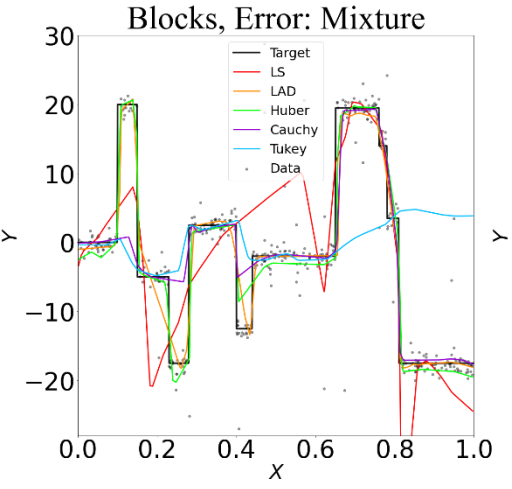
Numerical examples

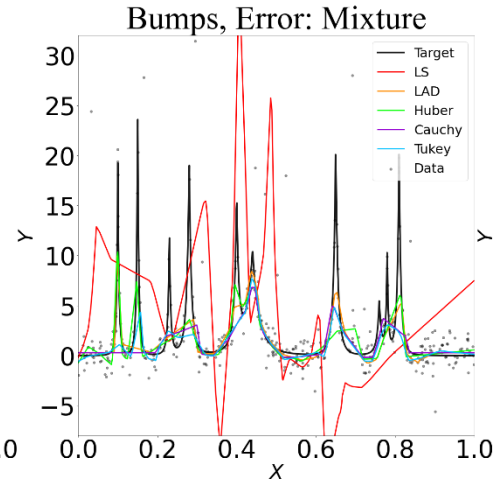


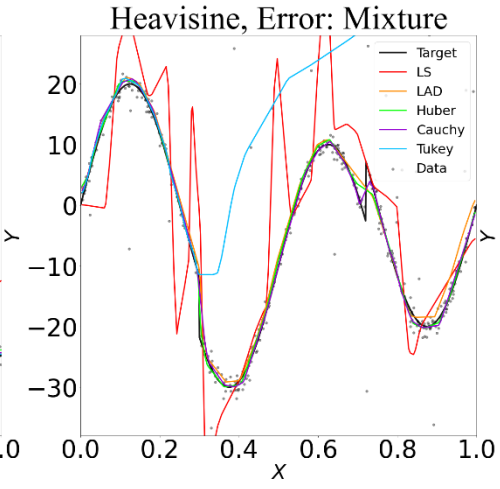


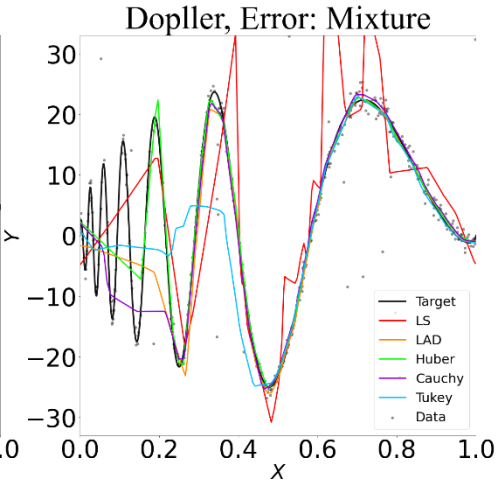






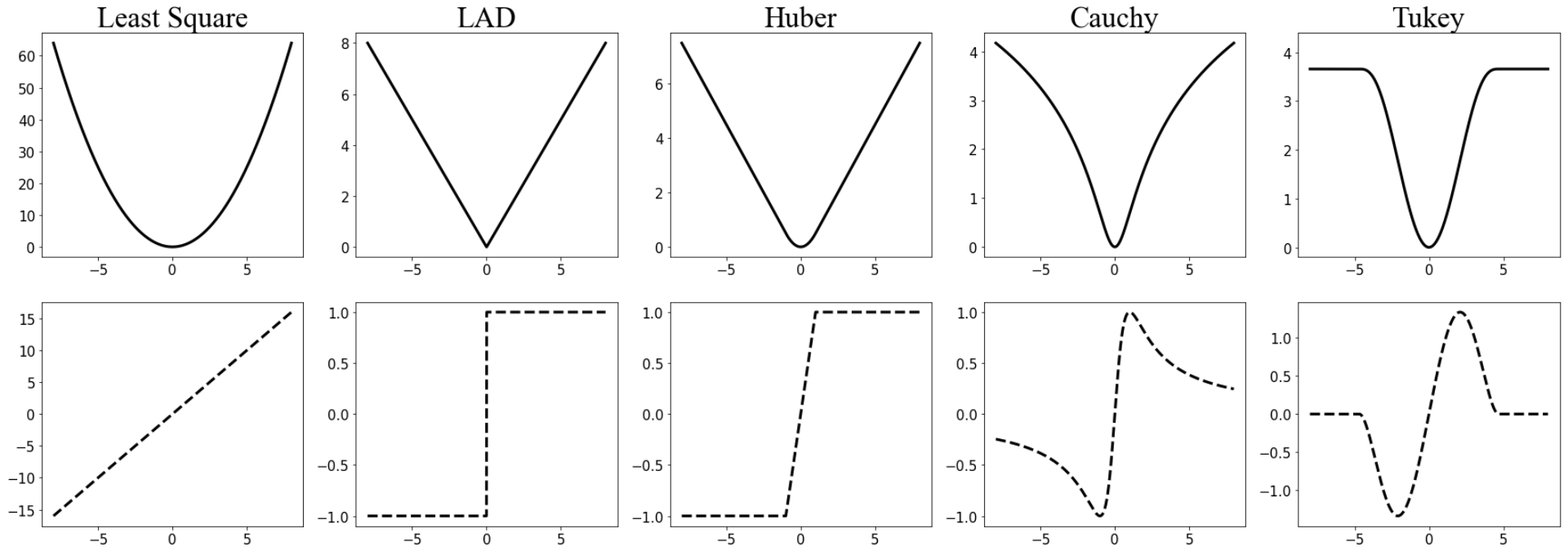






Deep Nonparametric Regression

Question: Then what is the target for robust regression?



Least square (LS): $\phi(a) = a^2$

Least absolute deviation (LAD): $\phi(a) = |a|$

Huber loss: $\phi(a) = \frac{a^2}{2} if\ |a| < \tau;\ \phi(a) = \tau|a| - \frac{\tau^2}{2}\ if\ |a| \geq \tau$ **for some** $\tau > 0$

Cauchy loss: $\phi(a) = log\ [1 + \kappa^2 a^2]$ **for some** $\kappa > 0$

Tukey loss: $\phi(a) = \frac{t^2\left\{1-\left[1-\left(\frac{a}{t}\right)^2\right]^3\right\}}{6}\ if\ |a| < t; \phi(a) = t^2/6\ otherwise$

Deep Nonparametric Regression

Recall the empirical risk minimization problem of robust regression

- **Data** $(X_i, Y_i), i = 1, \dots, n$.
- **To find a network** $f(x; \theta)$ **such that**

$$\frac{1}{n}\sum \phi(Y_i - f(X_i; \theta))$$

is minimized over

$$\mathcal{F} = \{f : f(x; \theta) \text{ is a neural network parameterized by } \theta \in \mathbb{R}^s\}.$$

- $\phi(\cdot): \mathbb{R} \to \mathbb{R}^+$ **for some (symmetric)** ϕ.

Deep Nonparametric Regression

Let's consider the Least Absolute Deviation (LAD)

- **Data $(X_i, Y_i), i = 1, \dots, n$ drawn from (X, Y).**
- **To find a network $f(x; \theta)$ such that $R_n(\theta) = R_n(f(\cdot, \theta)) = \frac{1}{n}\sum |Y_i - f(X_i; \theta)|$ is minimized over $\theta \in \mathbb{R}^s$ or over $f \in \mathcal{F}$.**
- **Let $R(f) = \mathbb{E}\|Y - f(X)\|_1$ be the risk of a function f. Then under mild condition, for each x**

$$f^*(x) = meadian(Y|X = x) = argmin_f \mathbb{E}\{\|Y - f(X)\|_1 | X = x\}.$$

- **Then the targets of the minimization of $R_n(f)$ is the conditional median of Y given X. In other words, LAD regression targets for "conditional median".**

Proof

- Let $R(f) = \mathbb{E}\|Y - f(X)\|_1$ be the risk of a function f. Then for each x

$$f^*(x) = meadian(Y|X = x) = argmin_f \mathbb{E}\{\|Y - f(X)\| \big| X = x\}.$$

- Suppose for each x, $\mathbb{E}[\|Y\|_1 | X = x] < +\infty$. Let $F_{Y|X=x}(\cdot)$ denote the conditional C.D.F of $Y|X = x$, then

$$\mathbb{E}\{\|Y - f(X)\|_1 | X = x\} = \int_{-\infty}^{f(x)} (f(x) - y) d\, F_{Y|X=x}(y) + \int_{f(x)}^{+\infty} \big(y - f(x)\big) d\, F_{Y|X=x}(y)$$

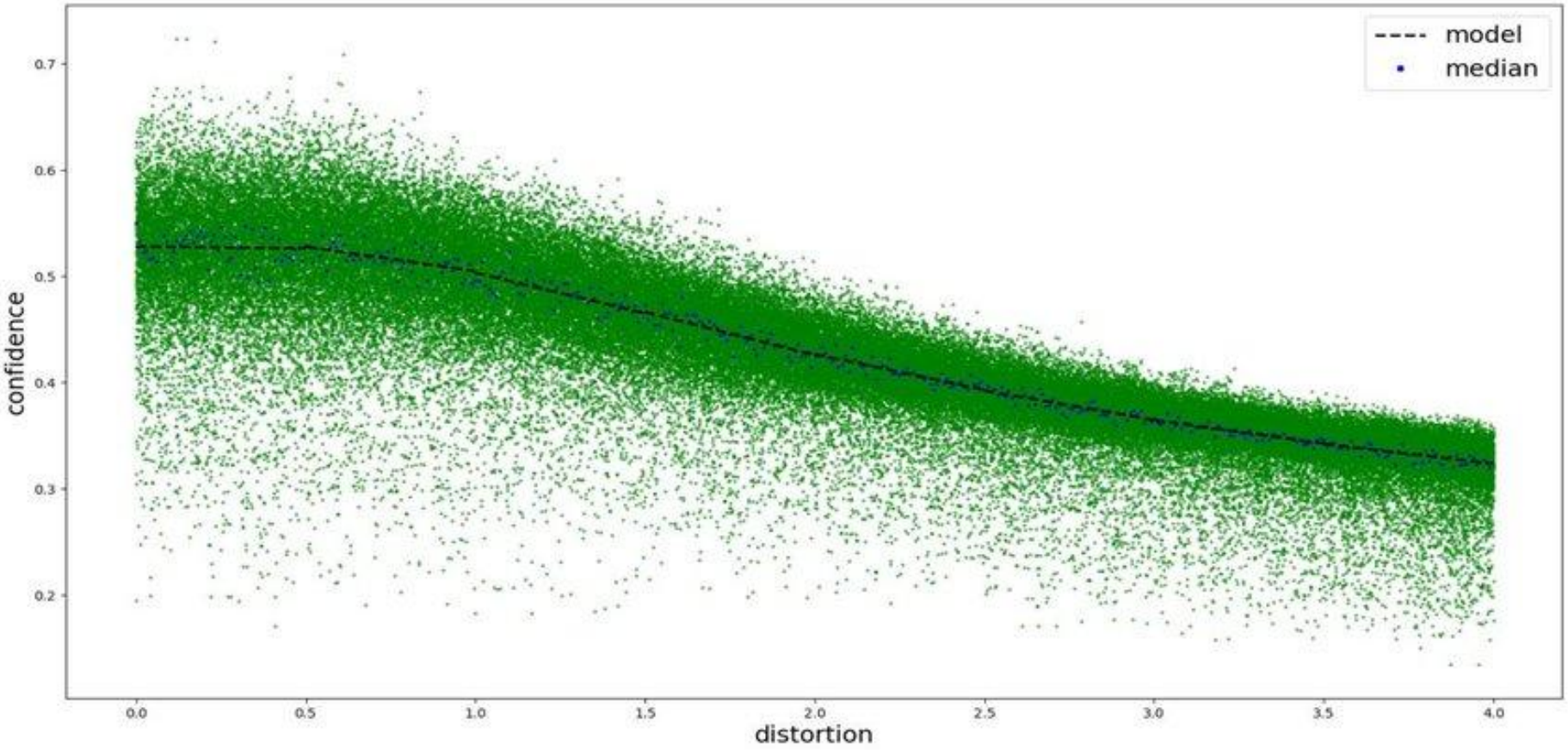
- Given f and x, then $f(x)$ is a scalar. We take derivative with respect to $f(x)$ in above quantity,
- $\frac{d}{df(x)} \mathbb{E}\{\|Y - f(X)\|_1 | X = x\} = \int_{-\infty}^{f(x)} 1\, d\, F_{Y|X=x}(y) + \int_{f(x)}^{+\infty} -1\, d\, F_{Y|X=x}(y)$

$$= F_{Y|X=x}(f(x)) - [1 - F_{Y|X=x}\big(f(x)\big)]$$

- Solve $\frac{d}{df(x)} \mathbb{E}\{\|Y - f(X)\|_1 | X = x\} = 0$, we have
- $F_{Y|X=x}(f^*(x)) = \frac{1}{2}$, this is equivalent to $f^*(x) = median(Y|X = x)$

Deep Nonparametric Regression

Least Absolute Deviation, regression for median



Source: https://doi.org/10.1117/12.2559457

Extension

- Note that in the last step, we have

$$F_{Y|X=x}(f^*(x)) = \frac{1}{2}$$

- Can we come up a loss function such that the minimizer $f^*(x)$ satisfies

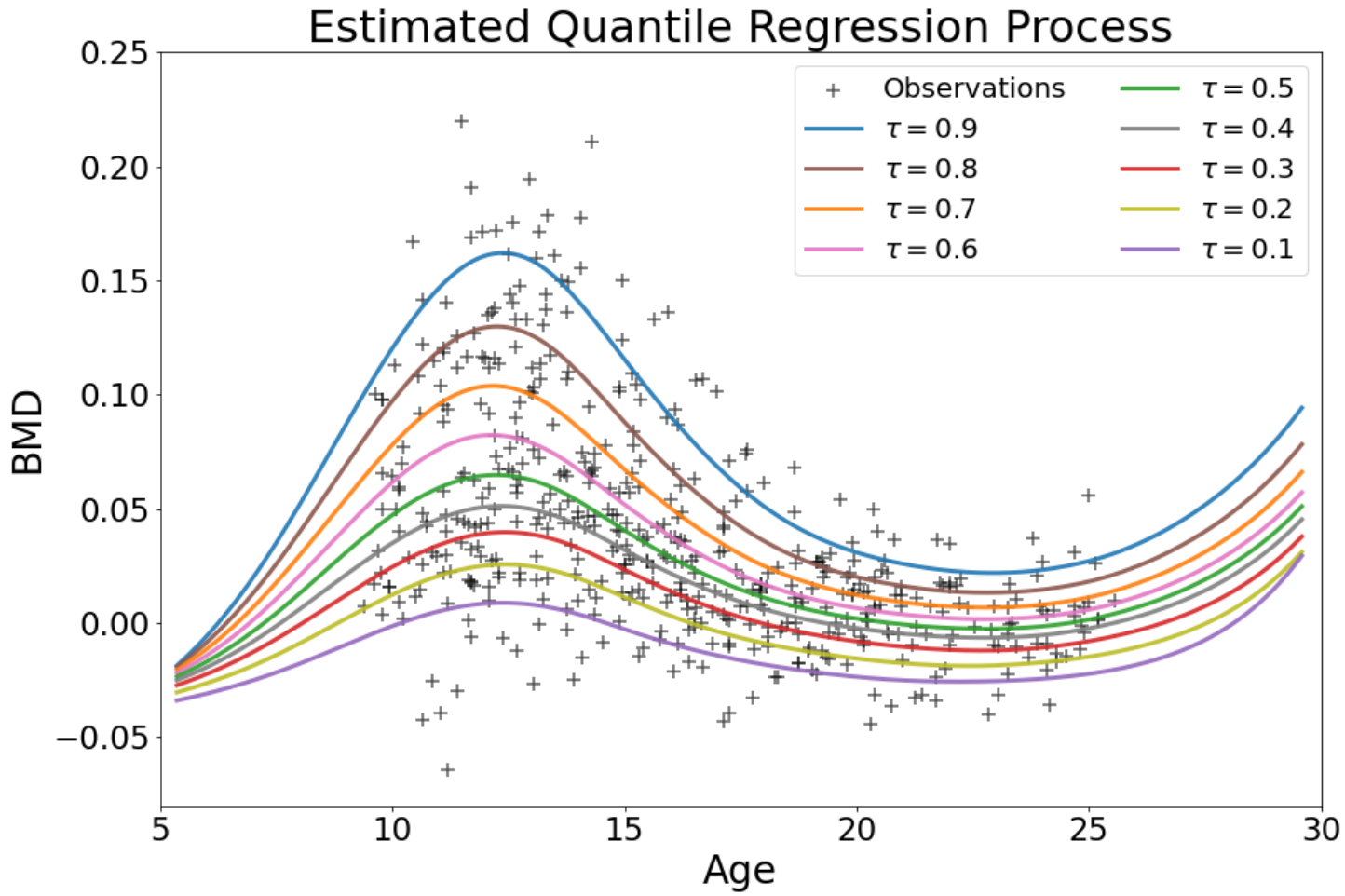
$$F_{Y|X=x}(f^*(x)) = \tau$$

for some $\tau \in (0, 1)$?

- Then $f^*(x) = F^{-1}_{Y|X=x}(\tau)$ will be the conditional τ –th quantile of $Y|X = x$.

- The answer is Yes.

Deep Quantile Regression



An example of deep quantile regression in BMD data set.

Source:https://arxiv.org/abs/2207.10442

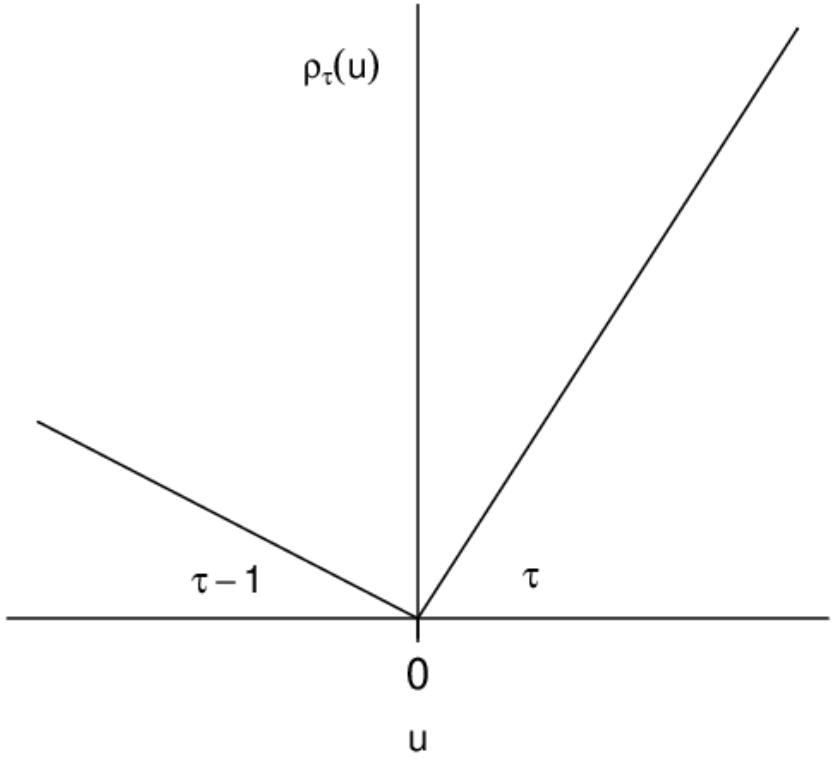
Check Loss function $\rho_\tau(a) = \big(\tau - I(a < 0)\big) \cdot a$

When $a > 0$,

$$\rho_\tau(a) = \big(\tau - I(a < 0)\big) \cdot a = \tau \cdot a$$

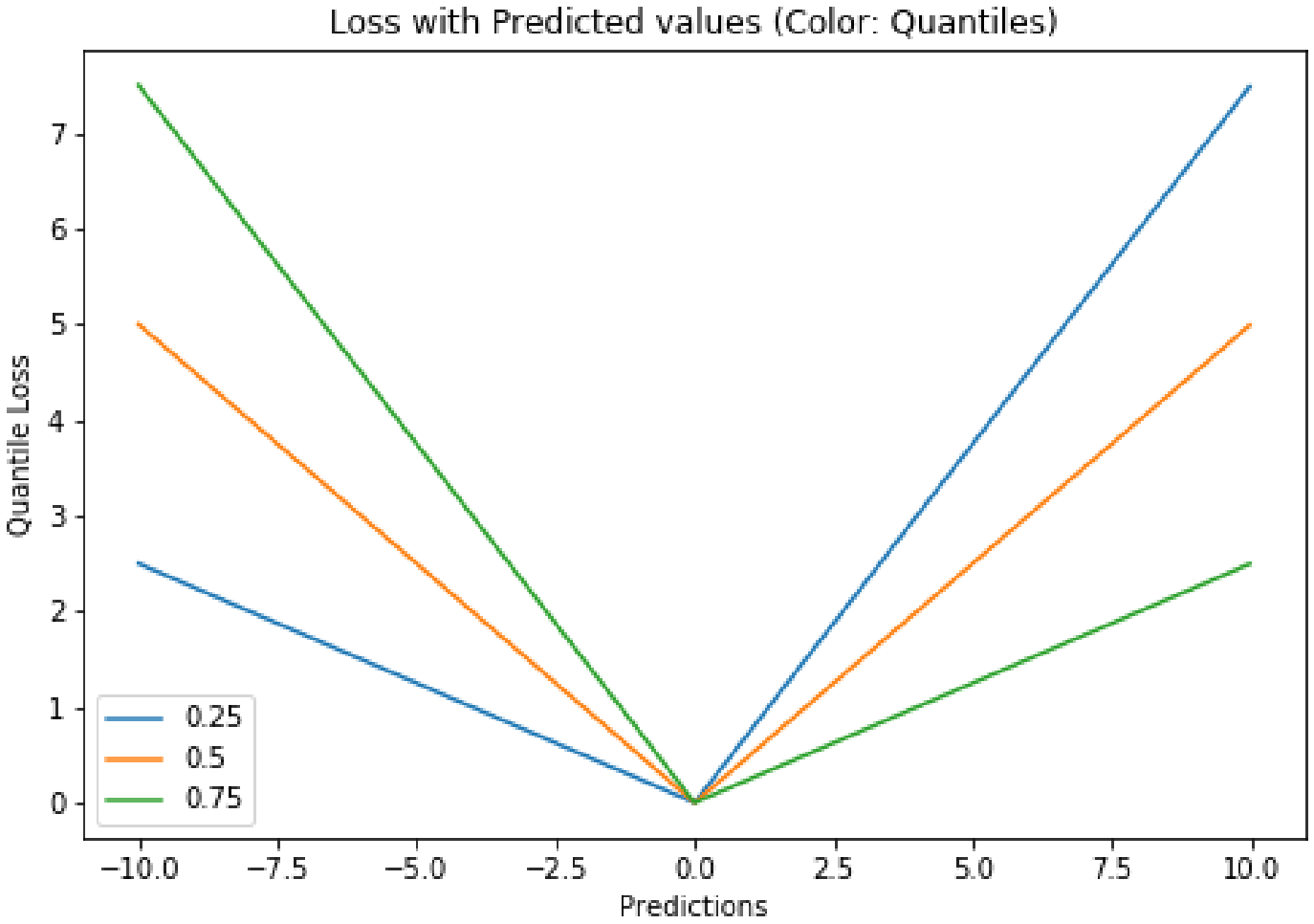
When $a \leq 0$,

$$\rho_\tau(a) = \big(\tau - I(a < 0)\big) \cdot a = (1 - \tau) \cdot (-a)$$



Deep Nonparametric Regression

Check Loss function $\rho_\tau(a) = \big(\tau - I(a < 0)\big) \cdot a$



$\tau = 0.25, 0.5, 0.75$

Deep Quantile Regression

- Data $(X_i, Y_i), i = 1, \dots, n$.

- Given $\tau \in (0, 1)$, to find a network $f(x; \theta)$ such that

$$\frac{1}{n}\sum \rho_\tau(Y_i - f(X_i; \theta))$$

is minimized over

$$\mathcal{F} = \{f : f(x; \theta) \text{ is a neural network parameterized by } \theta \in \mathbb{R}^s\}.$$

Deep Quantile Regression

- Let $R(f) = \mathbb{E}\rho_\tau(Y - f(X))$ be the risk of a function f.
- Then for each x

$$f^*(x) = \mathrm{argmin}_f \mathbb{E}\{\rho_\tau(Y - f(X)|X = x\}$$

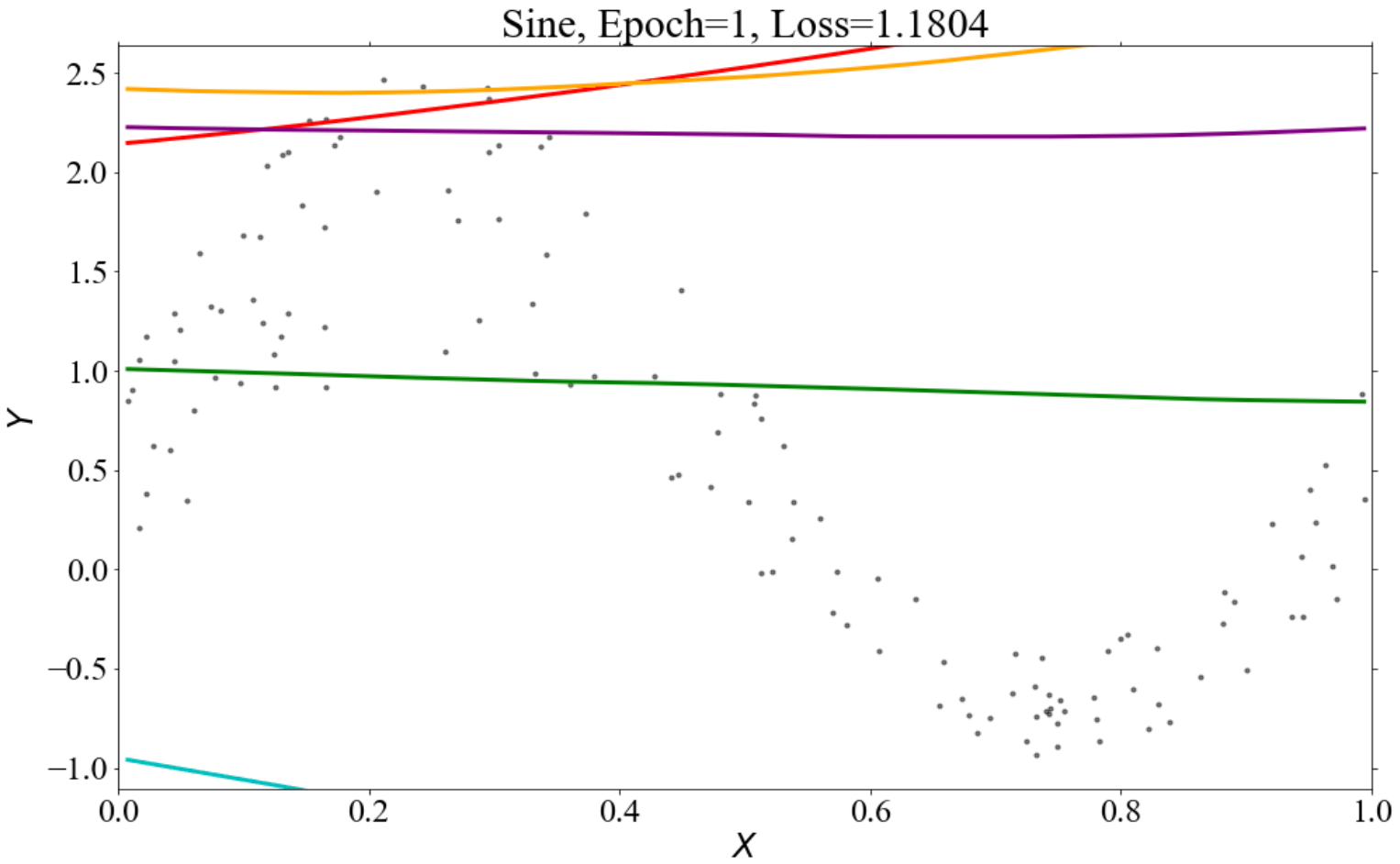
is the conditional τ–th quantile of Y given $X = x$.

And f^* is the minimizer of $R(f)$.

- Try to give the needed condition such that above claim holds. And prove it!

Deep Nonparametric Regression

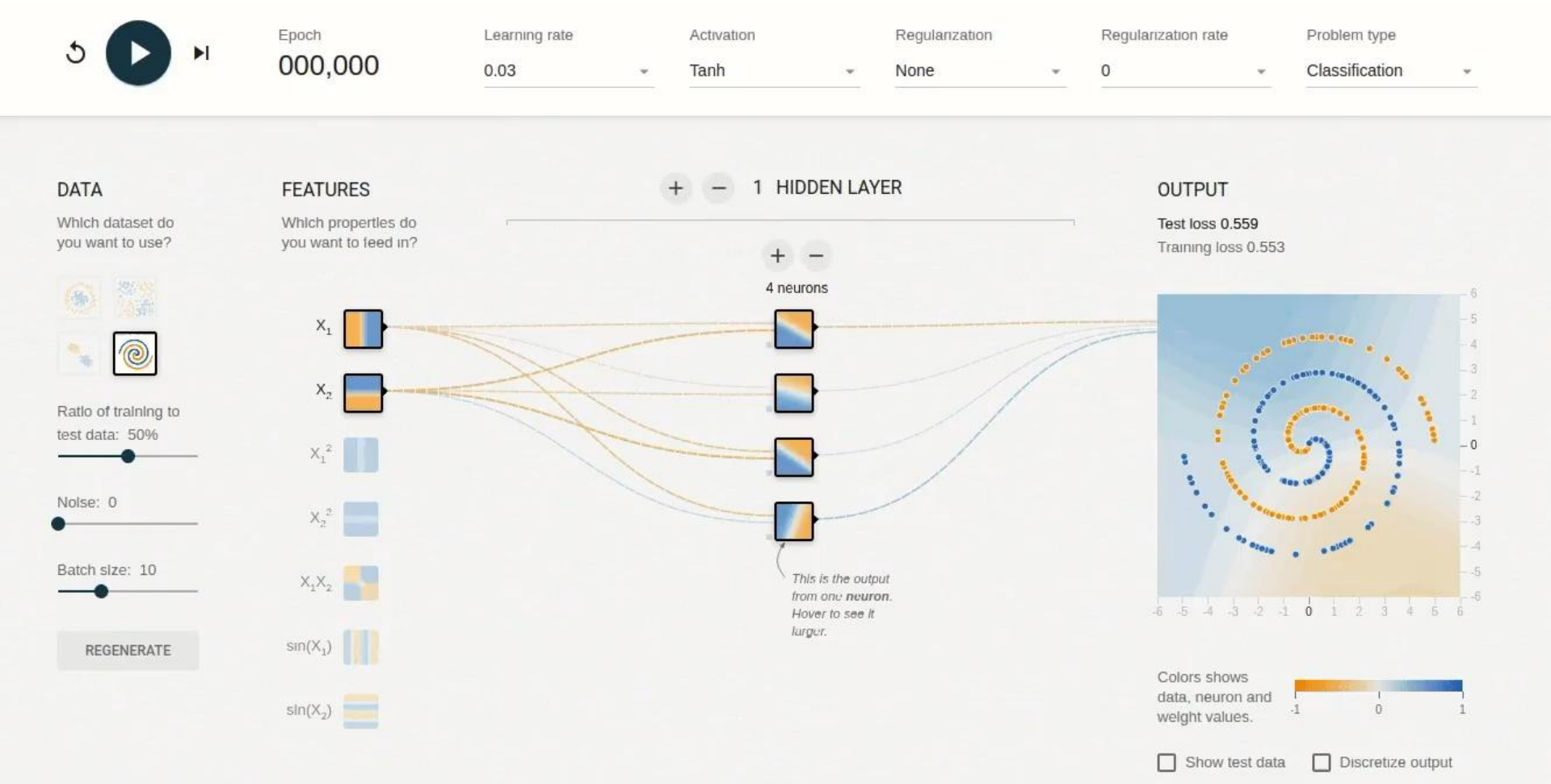
Deep Quantile Regression



Source: https://arxiv.org/abs/2107.04907

Deep Nonparametric Regression

TensorFlow Playground : https://playground.tensorflow.org/



Nonparametric Regression vs Parametric Regression

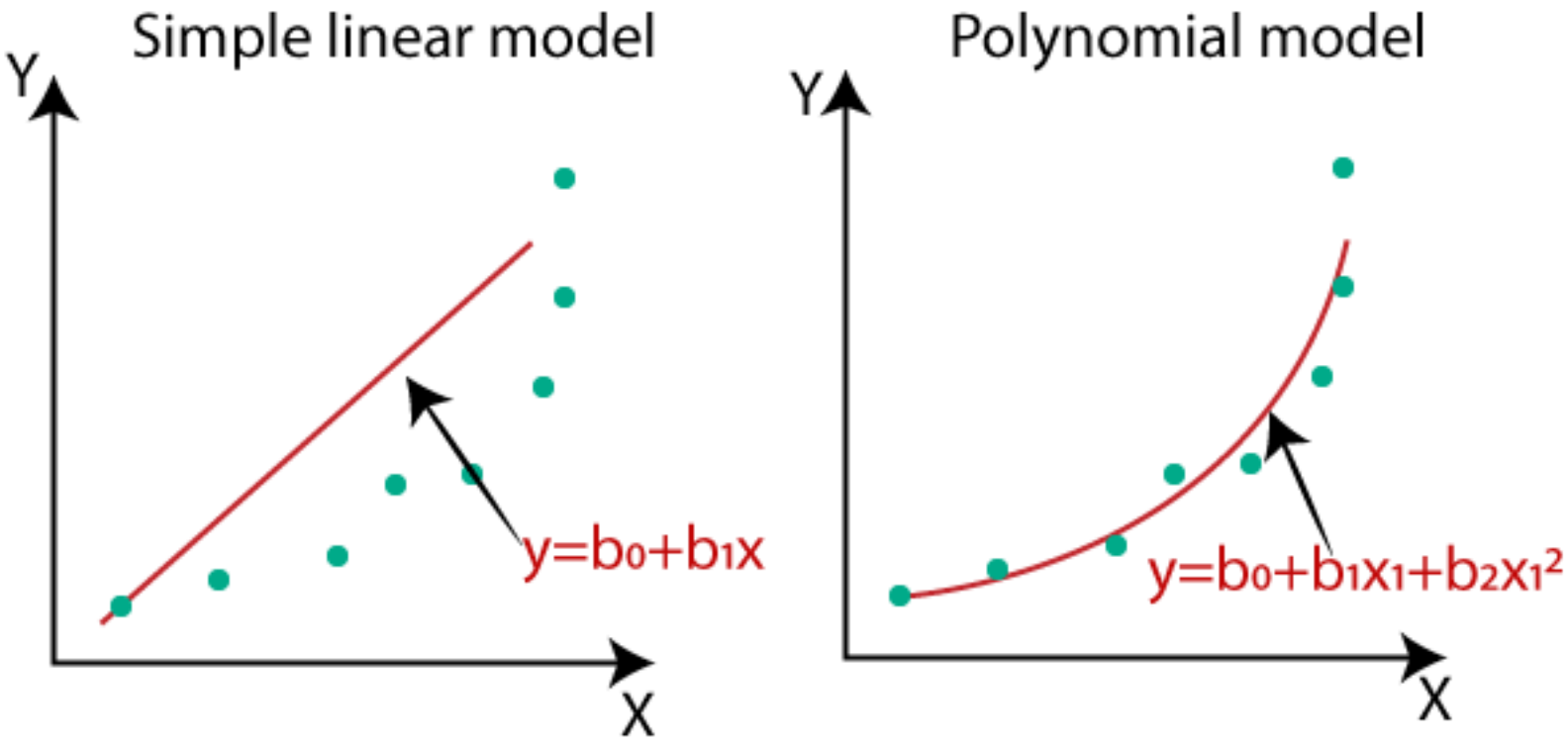
- **Linear models**, **generalized linear models**, and **nonlinear models** are examples of parametric regression models where we assume a function form that describes the relationship between the response and explanatory variables.

- In many situations, that relationship is not known. **Nonparametric regression** differs from parametric regression in that the shape of the functional relationships between the response (dependent) and the explanatory (independent) variables are not predetermined but can be adjusted to capture unusual or unexpected features of the data.

- When the relationship between the response and explanatory variables is known, parametric regression models should be used. If the relationship is unknown and nonlinear, nonparametric regression models should be used.

Nonparametric Regression vs Parametric Regression

- **Parametric regression models:** can be Linear and Nonlinear

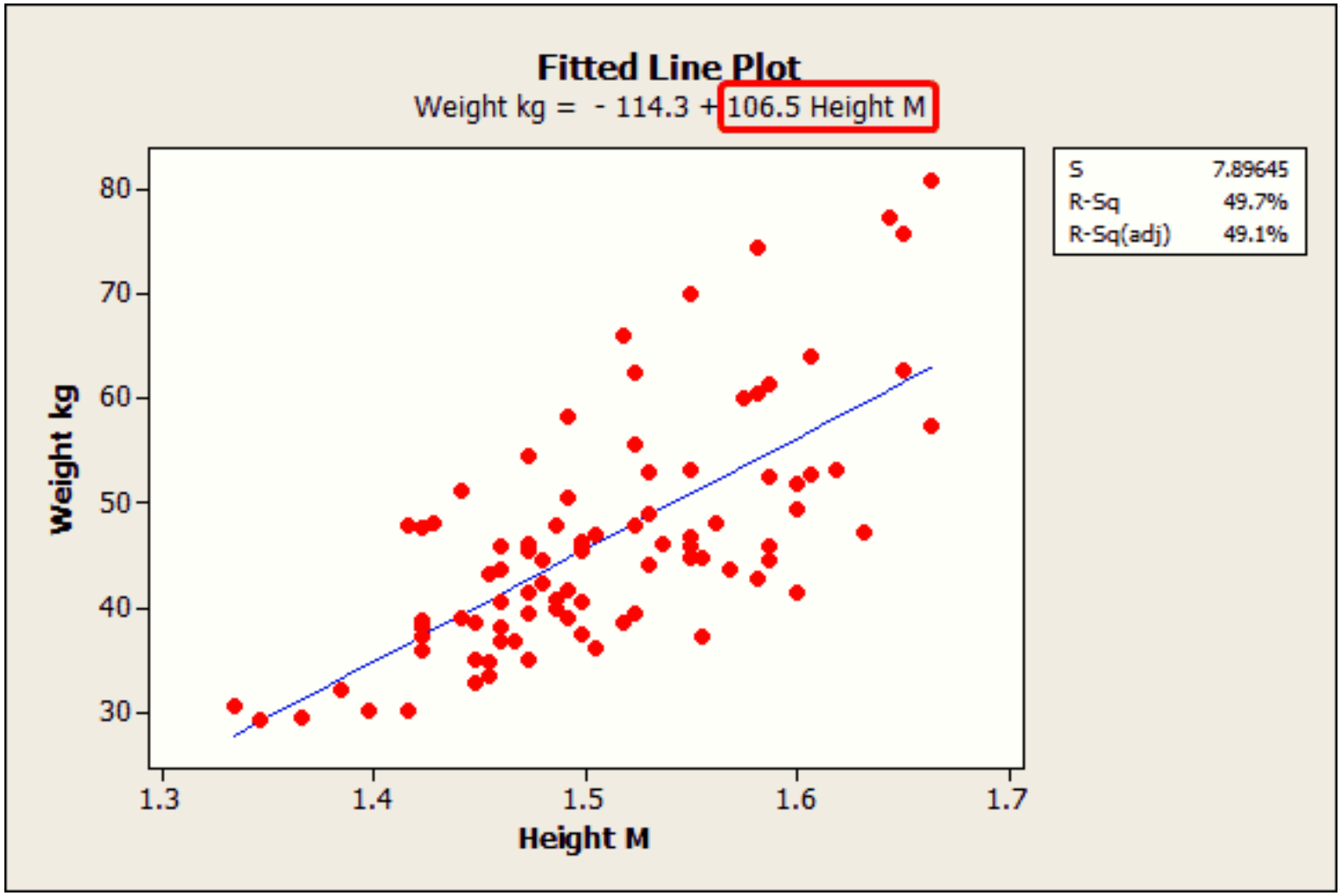


Source: https://www.javatpoint.com/machine-learning-polynomial-regression

Assume the function form that describes the relationship between the response and explanatory variables. Just estimate the parameters.

Nonparametric Regression vs Parametric Regression

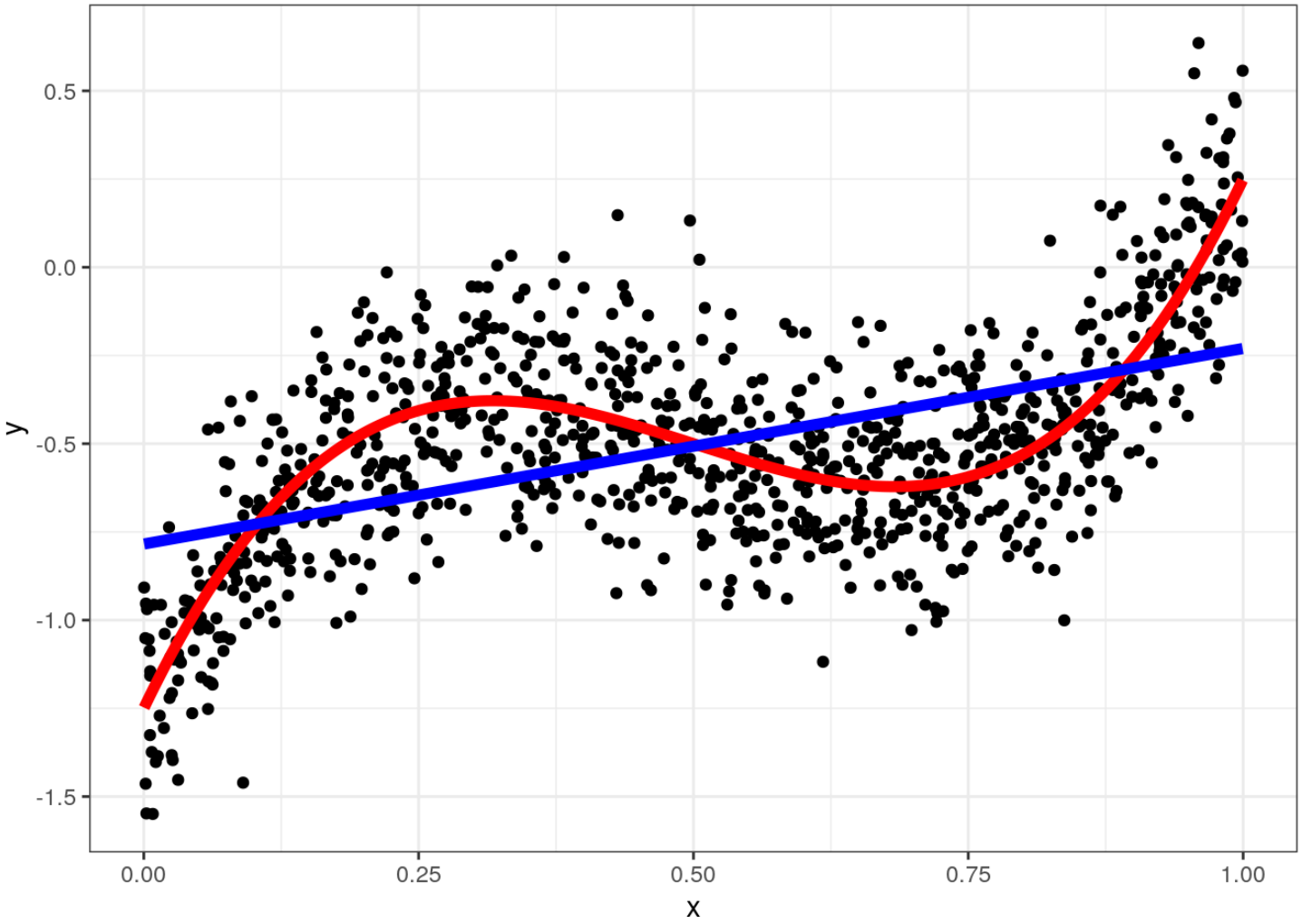
- **Parametric regression models** : Prediction and Inference



Source:https://statisticsbyjim.com/regression/interpret-coefficients-p-values-regression/

Nonparametric Regression vs Parametric Regression

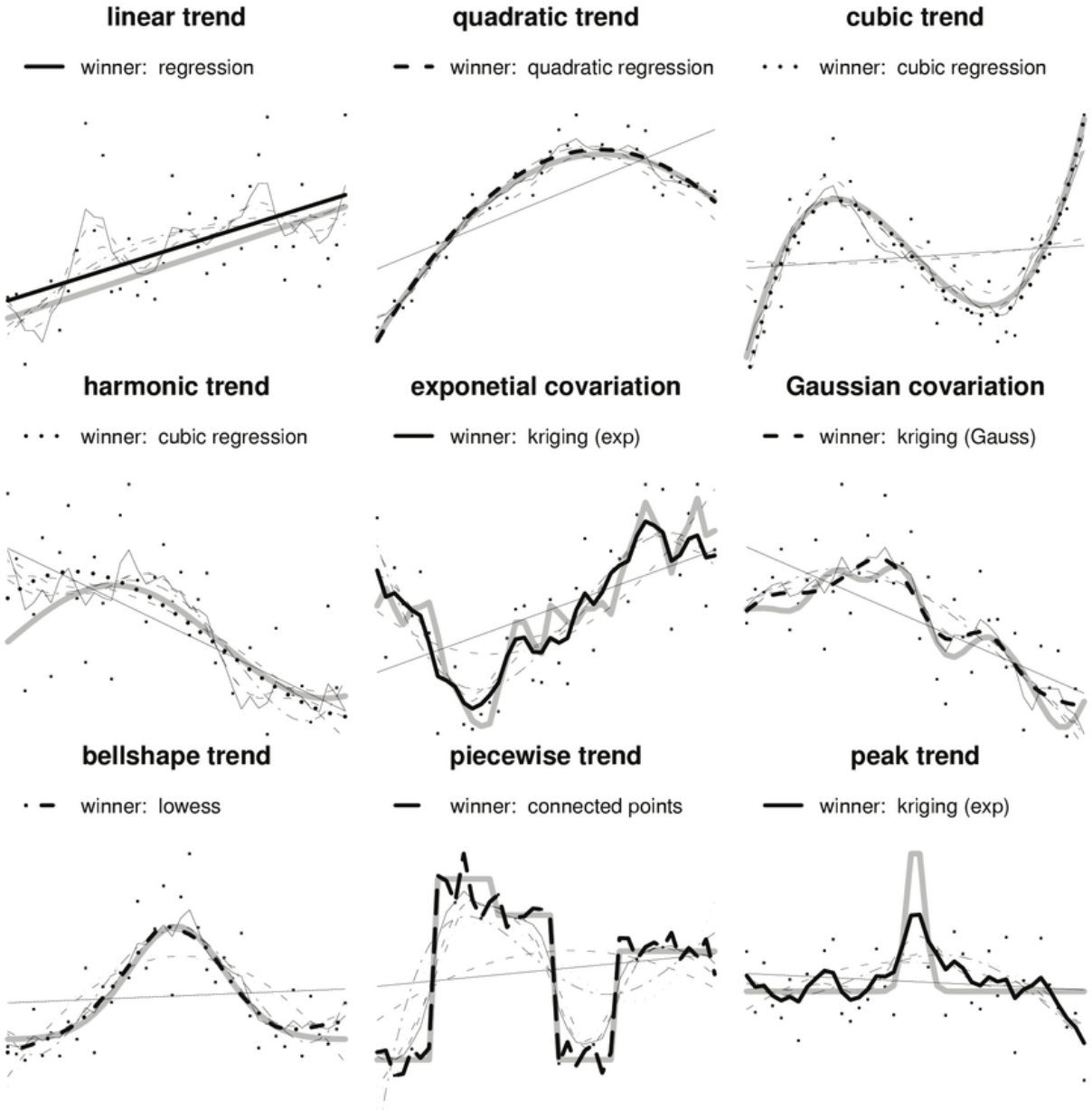
- **Parametric regression models** : May perform badly if model is misspecified



Source: http://econ21130.lamadon.com/np-regression.html

Nonparametric Regression vs Parametric Regression

- **Nonparametric regression models** : Flexible, data-driven



Source:https://www.researchgate.net/publication/251001544_Bayesian_Automating_Fitting_Functions_for_Spatial_Predictions